



PIF DOCUMENTATION

PIF (midterm 2)

Abstract

This documentation is writing about the entire PIF project on how to recreate it step-by-step

Christian Bartolé

2026

Table of Contents

LAB	5
Raspberry PI installation & configuration	5
Installation	5
VNC viewer	9
Configure static IP:	14
Enable I ² C interface:	15
Docker installation	17
Screen orientation	20
Disable on screen keyboard (optional)	24
Create folder /opt/station & /opt/station/www & copy files	24
Create a docker container	25
Configure autostart	26
Create folder /tmp/station	29
Redirect sensor readings to result.csv	29
MySQL setup	30
Mysql Database	30
Conceptual data model	33
Logical data model	33
Webserver	34
Notes:	34
Apache installation	34
MariaDB installation	34
PHP installation	34
phpMyAdmin installation	34
phpMyAdmin configuration	35
Install openssl and create self-signed SSL certificate	37
Redirect HTTP traffic to HTTPS	42
Data transfer	43
Config.toml	43
Receiver php script	43
Autostart configuration	46

Measurements file	47
Backup server	53
Allow external MySQL database access & restricted account	53
Create a new virtual machine	53
Export database setup.....	55
Crontab.....	56
Workshop.....	57
PVC plates assembly	57
Display plate.....	57
The base plate:	60
Plate assembly	61
PCB assembly	64
PVC plates assembly	68
Cable repair	73
Starting note:	73
Uninsulate wires	73
Wire stripping	75
Connect each wire back together	76
Cable insulation.....	77
Shielding tips	77
Sensors 1	78
Gas sensor	78
Bit I2C converter	79
Sensors 2.....	80
Biometric sensor.....	80
Gyroscope sensor	81
Sensor programming (BME280 / HW-611)	82
Sensor programming (BH1750)	84
Sources (for sprint Sensors 2):	86
Assembly	87
Material.....	87
Assembly (wooden box).....	88

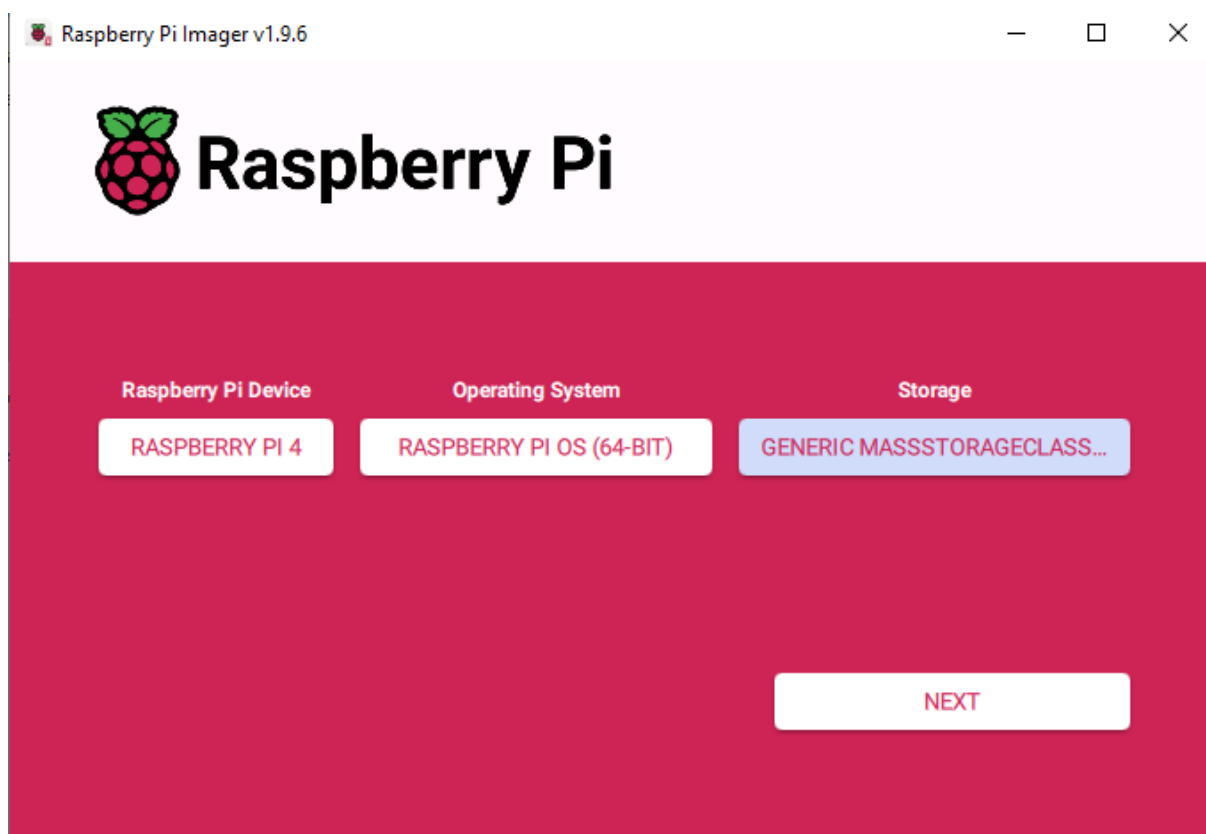
Assembly (button)	92
LED strip	97
Preparing LED strip.....	97
Putting the LED strip in place	99
Wire the LED strip to the board.....	100

LAB

Raspberry PI installation & configuration

Installation

After downloading, installing and executing the RasperryPI imager you need to select the correct model of the rasperry (Raspberry PI 4 Model 8 in this case), selecting the default & recommended Raspberry PI OS (64-BIT) & choose the correct drive (micro sd card in this case)



For the config of the raspberry you need to set a hostname which was in my case “PI<iam_code>” (“PI” was added by myself to mainly differentiate that it is a raspberrypi).

Configure a username & password, a network

OS Customisation

General

Services

Options

☒ Set hostname:

PIbarch584

.local

☒ Set username and password

Username:

barch584

Password:

.....

☒ Configure wireless LAN

SSID:

LPEM_B13

Password:

.....

☐ Hidden SSID

Wireless LAN country:

LU

☒ Set locale settings

Time zone:

Europe/Luxembourg

Keyboard layout:

ge

CANCEL

SAVE

In the services tab enable SSH & select **Use password authentication**

OS Customisation

General

Services

Options

☒ Enable SSH

☒ Use password authentication

☐ Allow public-key authentication only

Set authorized_keys for 'barch584':

DELETE KEY

RUN SSH-KEYGEN

ADD SSH KEY

In the **Options** don't change anything

OS Customisation

General

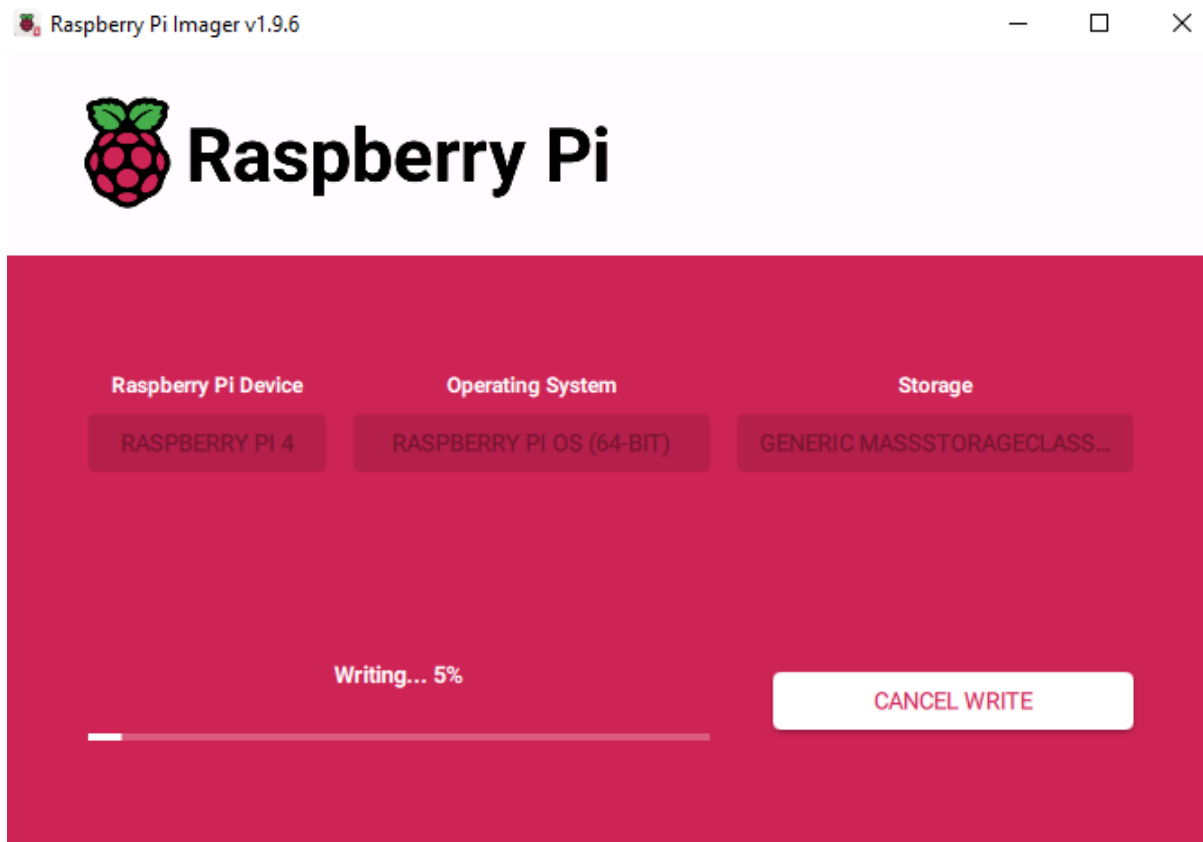
Services

Options

☐ Play sound when finished

☒ Eject media when finished

☒ Enable telemetry



Test if it works firstly with a cmd. Type in **ssh <username>@<hostname>** which was for my case **ssh barch584@PIbarch584**

```
cmd Select barch584@PIbarch584: ~
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

C:\Users\BarCh584.LTPEMDOMAIN>ssh barch584@PIbarch584.local
ssh: Could not resolve hostname pibarch584.local: No such host is known.

C:\Users\BarCh584.LTPEMDOMAIN>ssh barch584@PIbarch584
The authenticity of host 'pibarch584 (fe80::7240:5a11:824a:8f75%15)' can't be established.
ED25519 key fingerprint is SHA256:MrKrbPqj6bTL3+bUCXi0Ucsy250BNBLDckvA0Vckmtg.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added 'pibarch584' (ED25519) to the list of known hosts.
barch584@pibarch584's password:
Linux PIbarch584 6.12.47+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.12.47-1+rpt1 (2025-09-16) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
barch584@PIbarch584:~ $
```


VNC viewer

To enable that you can access the desktop of the raspberry (with VNC viewer in this case) type in the cmd prompt (connected to the raspberry PI via SSH) **sudo raspi-config**

```
barch584@PIbarch584:~ $ sudo raspi-config
```

Select 3. Interface Options

```
ch barch584@PIbarch584: ~
Raspberry Pi 4 Model B Rev 1.5, 1GB

Raspberry Pi Software Configuration Tool (raspi-config)

1 System Options      Configure system settings
2 Display Options     Configure display settings
3 Interface Options   Configure connections to peripherals
4 Performance Options Configure performance settings
5 Localisation Options Configure language and regional settings
6 Advanced Options   Configure advanced settings
8 Update              Update this tool to the latest version
9 About raspi-config  Information about this configuration tool

<Select>                                <Finish>
```

Select I3 VNC

```
ch barch584@PIbarch584: ~

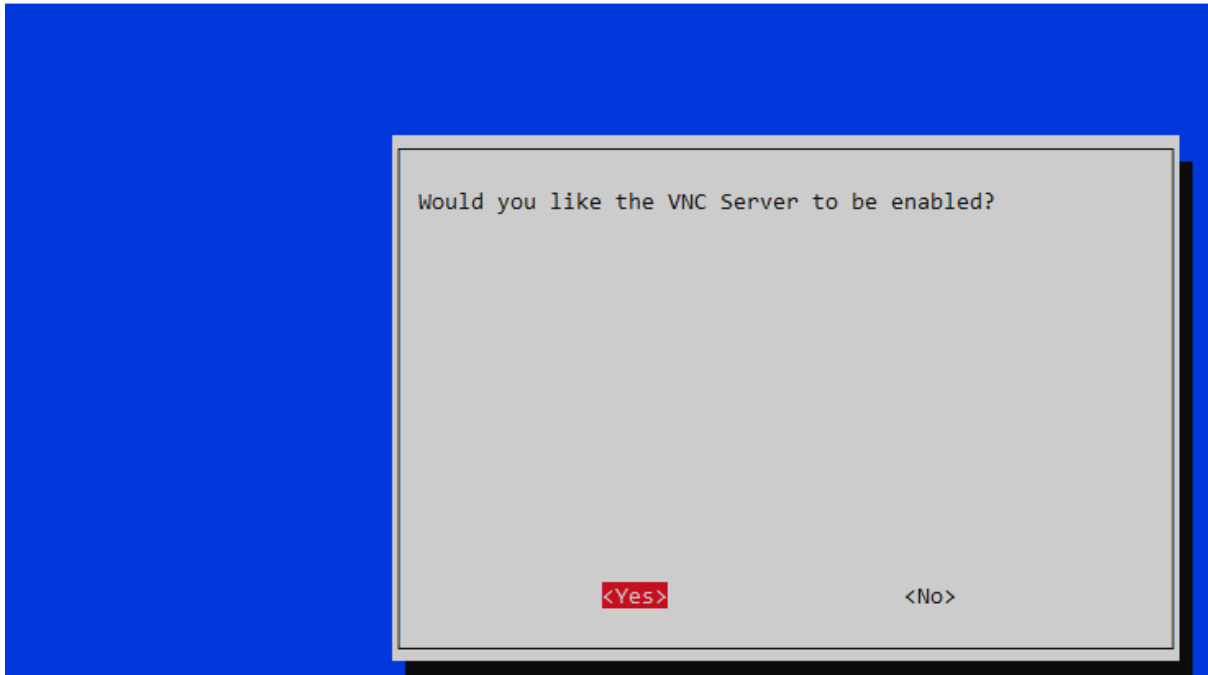
Raspberry Pi Software Configuration Tool (raspi-config)

I1 SSH      Enable/disable remote command line access using SSH
I2 RPi Connect Enable/disable Raspberry Pi Connect
I3 VNC      Enable/disable graphical remote desktop access
I4 SPI      Enable/disable automatic loading of SPI kernel module
I5 I2C      Enable/disable automatic loading of I2C kernel module
I6 Serial Port Enable/disable shell messages on the serial connection
I7 1-Wire   Enable/disable one-wire interface

<Select>                                <Back>
```

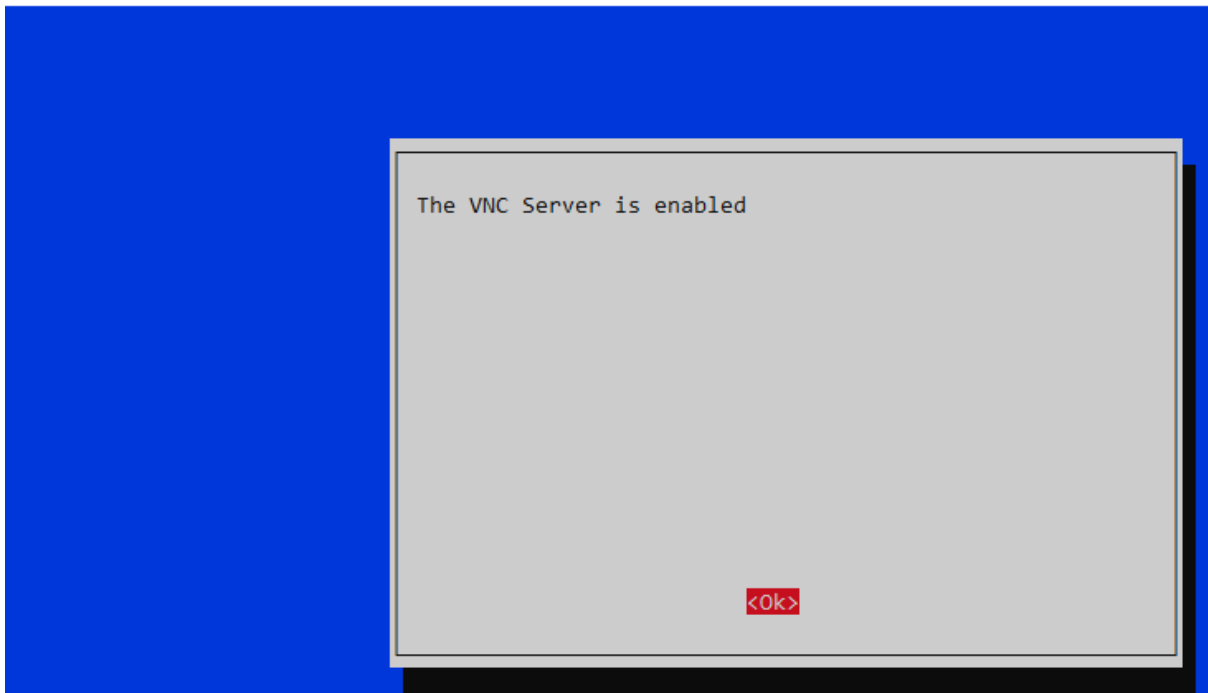
Select **Yes**

```
Ctrl barch584@PIbarch584: ~
```



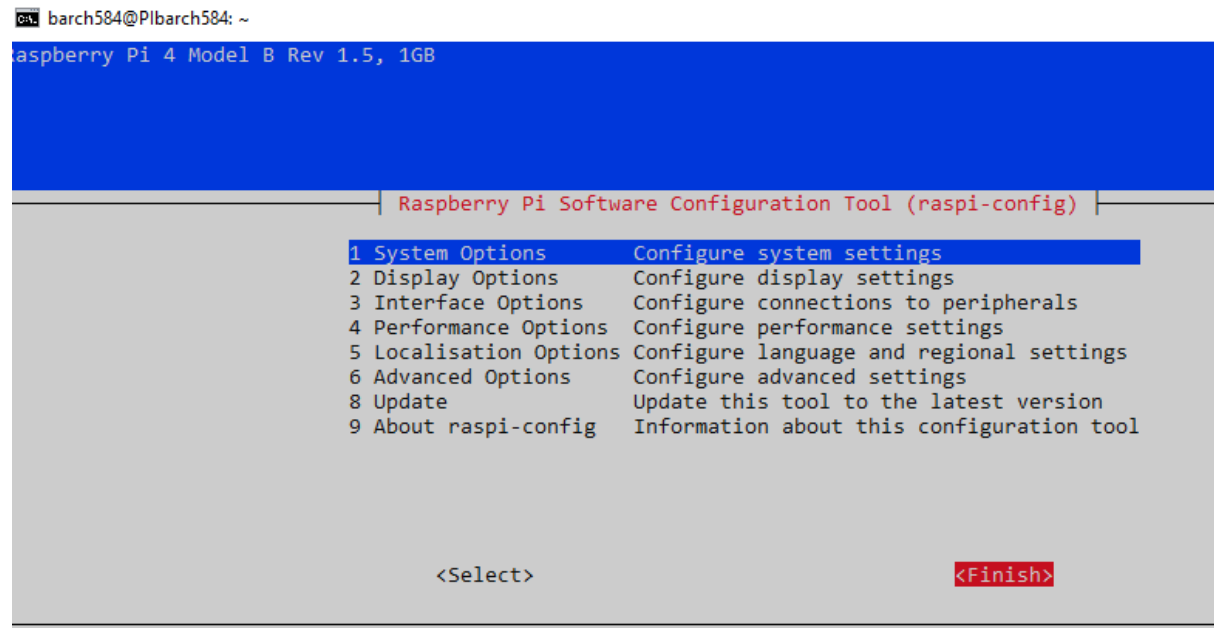
If successful it should display:

```
Ctrl barch584@PIbarch584: ~
```



With the **Tab** key you can go to **Finish**

```
cat barch584@PIbarch584: ~
raspberrypi Pi 4 Model B Rev 1.5, 1GB
```



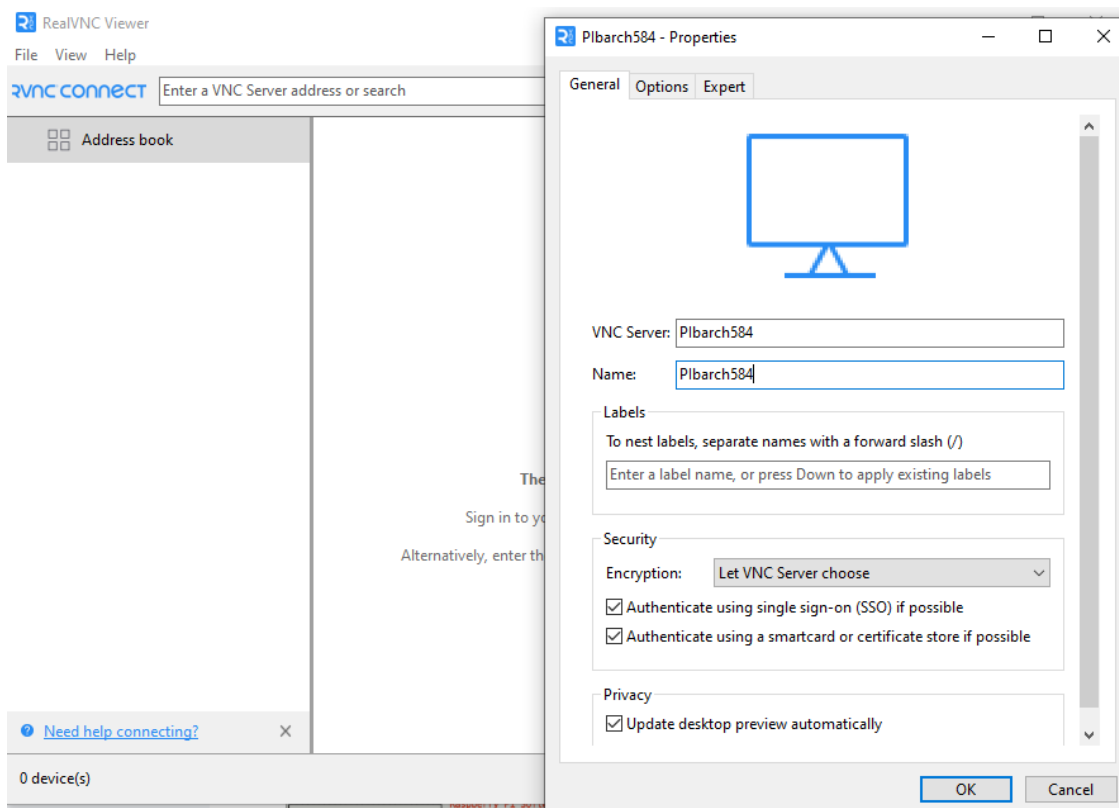
Raspberry Pi Software Configuration Tool (raspi-config)	
1 System Options	Configure system settings
2 Display Options	Configure display settings
3 Interface Options	Configure connections to peripherals
4 Performance Options	Configure performance settings
5 Localisation Options	Configure language and regional settings
6 Advanced Options	Configure advanced settings
8 Update	Update this tool to the latest version
9 About raspi-config	Information about this configuration tool

<Select> **<Finish>**

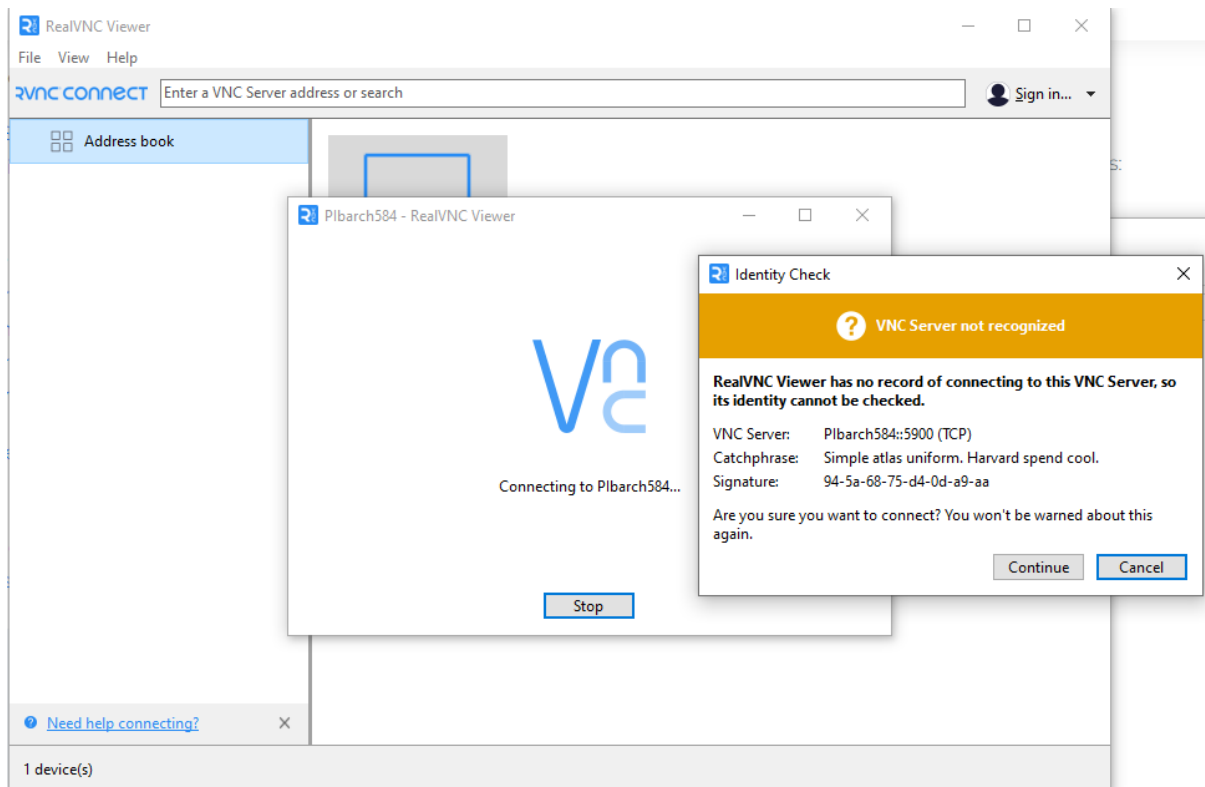
Now connect to it via VNC viewer:

(Open VNC viewer), Go to **File > New connection**

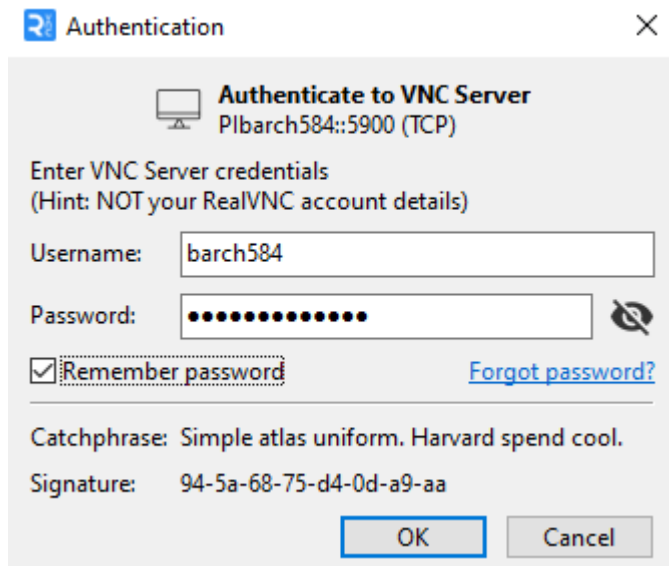
Type in the hostname of the PI in the **VNC Server**: give it a name of your liking in **Name** and select **OK**



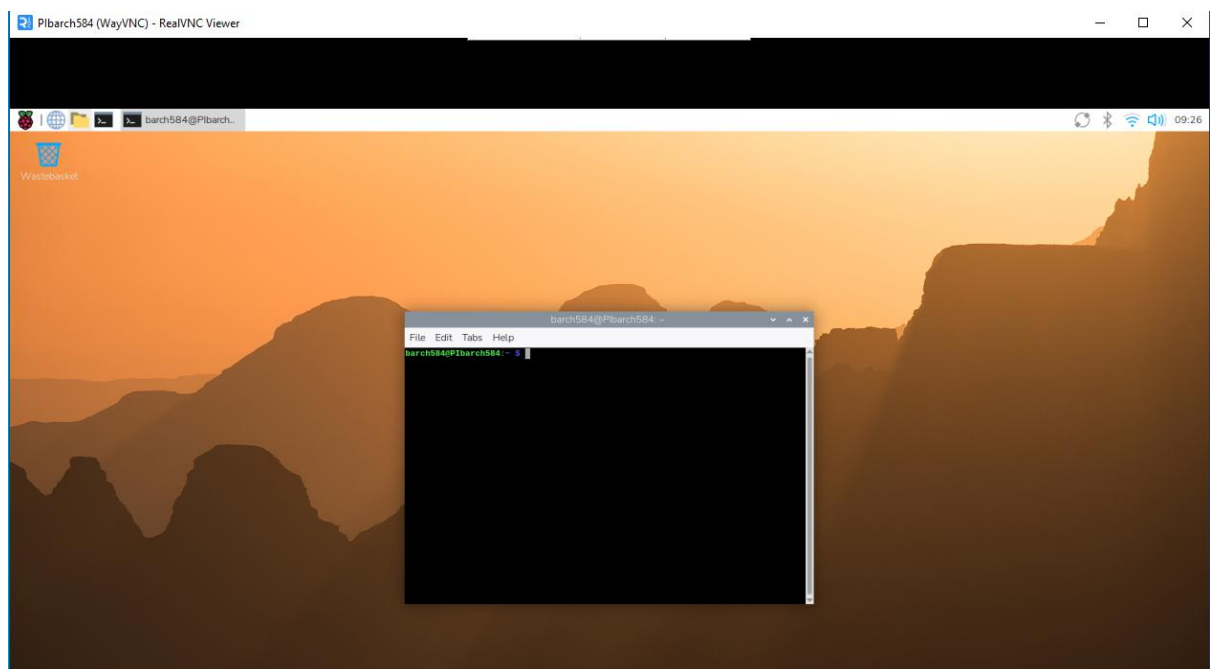
Double click the now created icon which has the name you entered earlier and if asked for the **Identity Check** window click **Continue**



Type in the **username & password** setup earlier in the configuration of the raspberry PI and you can also select **Remember password** (not mandatory but practical)



Now it is successfully connected via VNC, and you see the desktop of the raspberry.



Configure static IP:

Open a terminal and type in **sudo nano /etc/dhcpd.conf** and add on a new empty line:

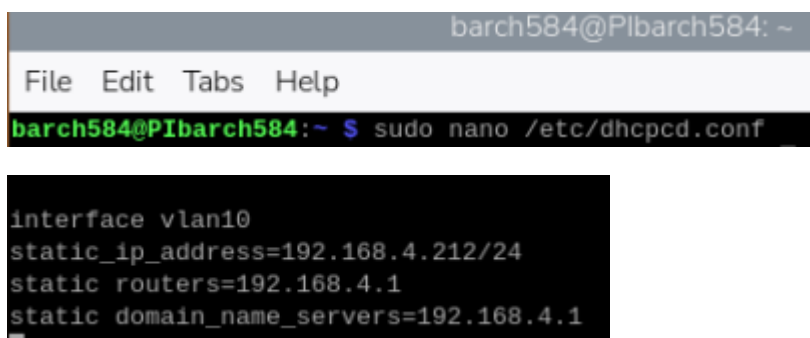
(Note that **vlan10** can have also a different number (it can also e.g. be 20, 30, or any other number & for the **static_ip_address** the last 3 digits are 212 because due to the PC number being 12 & the first digit was choosed by the organizers)

interface vlan10

static ip_address=192.168.4.212/24

static routers=192.168.4.1

static domain_name_servers=192.168.4.1



```
barch584@PIbarch584: ~  
File Edit Tabs Help  
barch584@PIbarch584:~ $ sudo nano /etc/dhcpd.conf  
  
interface vlan10  
static_ip_address=192.168.4.212/24  
static routers=192.168.4.1  
static domain_name_servers=192.168.4.1
```

Next step is to install the dhcpcd library with the command

sudo apt install dhcpcd5 -y

```
barch584@PiBarch584:~$ sudo apt install dhcpcd5 -y
Installing:
  dhcpcd5

Installing dependencies:
  dhcpcd

Suggested packages:
  dhcpcd-gtk

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 6
  Download size: 25.3 kB
  Space needed: 64.5 kB / 22.8 GB available

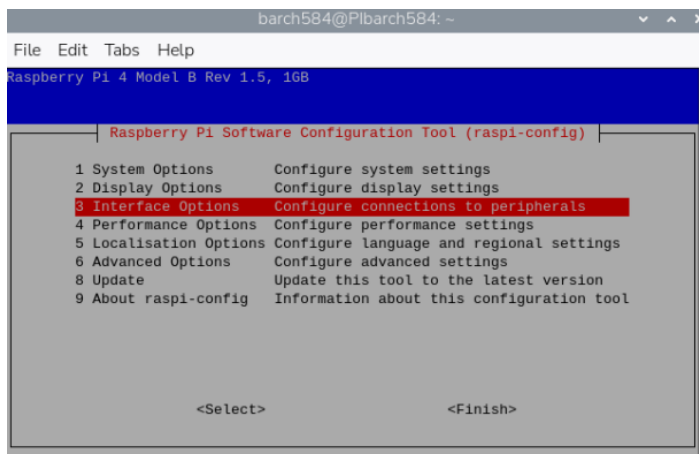
Get:1 http://deb.debian.org/debian trixie/main arm64 dhcpcd all 1:10.1.0-11 [13.9 kB]
Get:2 http://deb.debian.org/debian trixie/main arm64 dhcpcd5 all 1:10.1.0-11 [11.4 kB]
Fetched 25.3 kB in 0s (253 kB/s)
Selecting previously unselected package dhcpcd.
(Reading database ... 120698 files and directories currently installed.)
Preparing to unpack .../dhcpcd_1%3a10.1.0-11_all.deb ...
Unpacking dhcpcd (1:10.1.0-11) ...
```

Lastly reboot the system by typing in **sudo reboot now**.

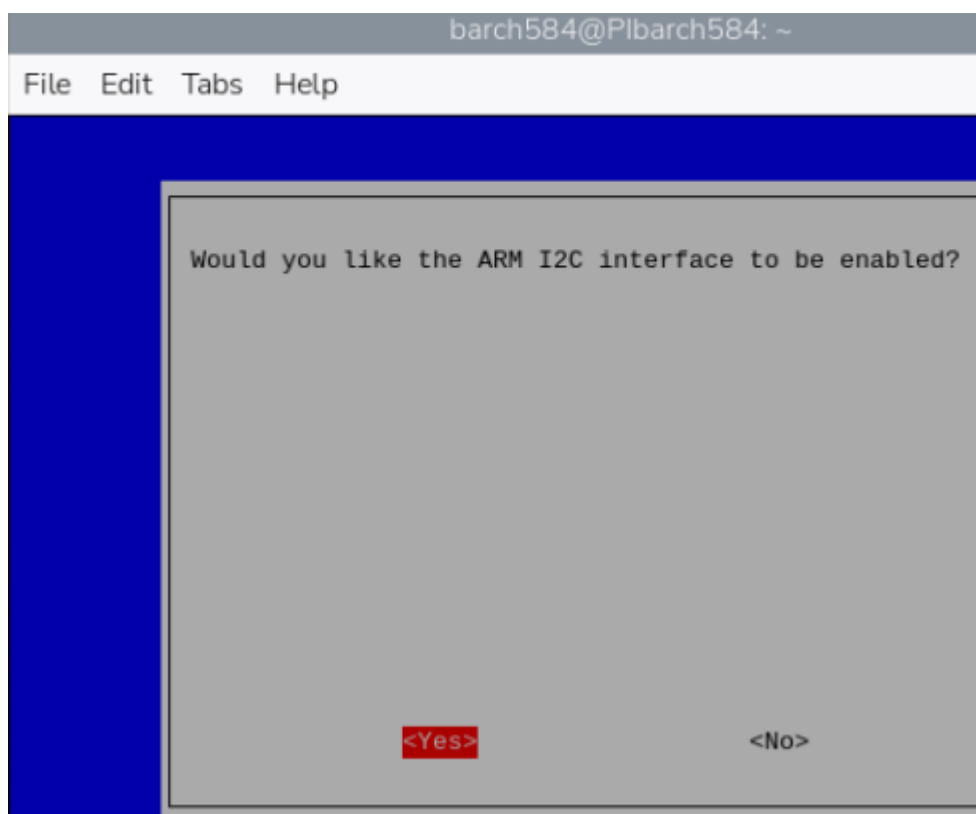
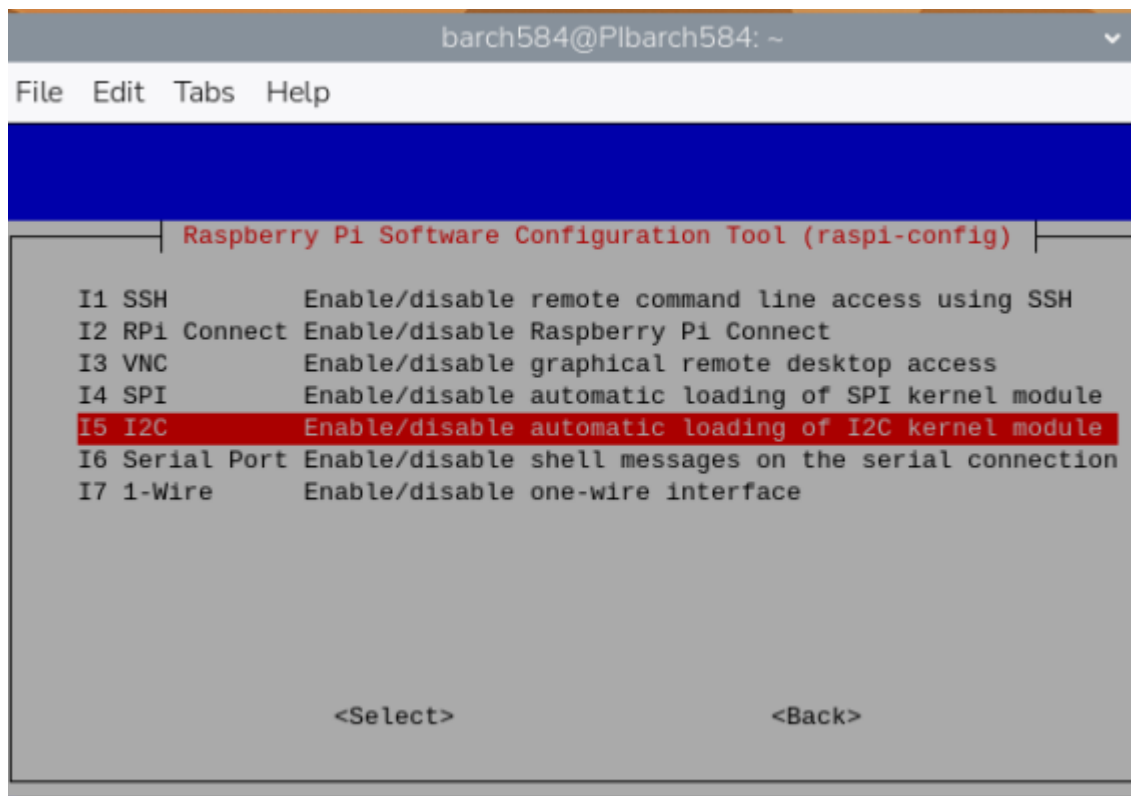
Enable I²C interface:

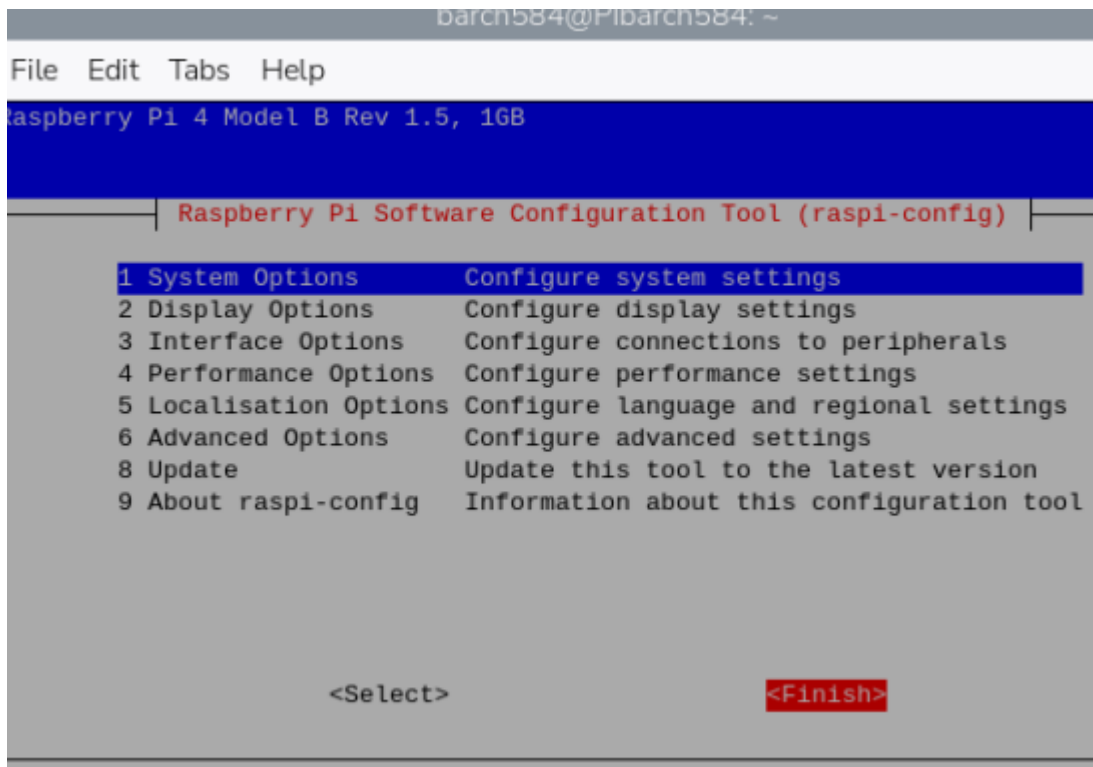
```
barch584@PiBarch584:~$ sudo raspi-config
```

Go to **3. Interface Options**



Go to **I5 I2C** and with the **Tab** key navigate to **Yes** to enable it and with **Tab** navigate to **Finish**

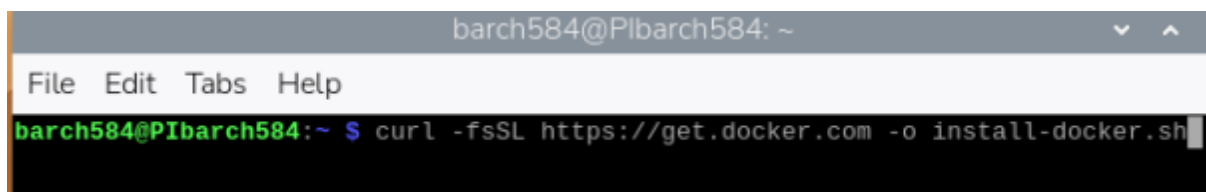




Docker installation

To install Docker in the Raspberry Pi, use the resource provided on get.docker.com, meaning firstly download it with the command

curl -fsSL <https://get.docker.com> -o install-docker.sh



Verify the content of the script

Cat install-docker.sh

(This is the beginning of the script)

```

barch584@PIbarch584:~ $ cat install-docker.sh
#!/bin/sh
set -e
# Docker Engine for Linux installation script.
#
# This script is intended as a convenient way to configure docker's package
# repositories and to install Docker Engine. This script is not recommended
# for production environments. Before running this script, make yourself familiar
# with potential risks and limitations, and refer to the installation manual
# at https://docs.docker.com/engine/install/ for alternative installation methods.
#
# The script:
#
# - Requires `root` or `sudo` privileges to run.
# - Attempts to detect your Linux distribution and version and configure your
#   package management system for you.
# - Doesn't allow you to customize most installation parameters.
# - Installs dependencies and recommendations without asking for confirmation.
# - Installs the latest stable release (by default) of Docker CLI, Docker Engine,
#   Docker Buildx, Docker Compose, containerd, and runc. When using this script
#   to provision a machine, this may result in unexpected major version upgrades
#   of these packages. Always test upgrades in a test environment before
#   deploying to your production systems.
# - Isn't designed to upgrade an existing Docker installation. When using the
#   script to update an existing installation, dependencies may not be updated
#   to the expected version, resulting in outdated versions.
#
# Source code is available at https://github.com/docker/docker-install/
#
# Usage
# =====
#
# To install the latest stable versions of Docker CLI, Docker Engine, and their
# dependencies:

```

Now verify the scripts steps to verify that it would execute properly:

(The `--dry-run` parameter specifies that the terminal will display all the steps it did when it executed the script)

sh install-docker.sh --dry-run

```

barch584@PIbarch584:~ $ sh install-docker.sh --dry-run
# Executing docker install script, commit: 7d96bd3c5235ab2121bcb855dd7b3f3f37128ed4
apt-get -qq update >/dev/null
DEBIAN_FRONTEND=noninteractive apt-get -y -qq install ca-certificates curl >/dev/null
install -m 0755 -d /etc/apt/keyrings
curl -fsSL "https://download.docker.com/linux/debian/gpg" -o /etc/apt/keyrings/docker.asc
chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=arm64 signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/"
apt-get -qq update >/dev/null
DEBIAN_FRONTEND=noninteractive apt-get -y -qq install docker-ce docker-ce-cli containerd.io
barch584@PIbarch584:~ $

```

And to finally install it run

sudo sh install-docker.sh

```
barch584@PIbarch584:~ $ sudo sh install-docker.sh
# Executing docker install script, commit: 7d96bd3c5235ab2121bcb855dd7b3f37128ed4
+ sh -c apt-get -qq update >/dev/null
+ sh -c DEBIAN_FRONTEND=noninteractive apt-get -y -qq install ca-certificates curl >/dev/null
+ sh -c install -m 0755 -d /etc/apt/keyrings
+ sh -c curl -fsSL "https://download.docker.com/linux/debian/gpg" -o /etc/apt/keyrings/docker.asc
+ sh -c chmod a+r /etc/apt/keyrings/docker.asc
+ sh -c echo "deb [arch=arm64 signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/debian" >/etc/apt/sources.list.d/docker.list
+ sh -c apt-get -qq update >/dev/null
+ sh -c DEBIAN_FRONTEND=noninteractive apt-get -y -qq install docker-ce docker-ce-cli containerd.io docker-compose-plugin
+ sh -c docker version
```

To verify that docker was successfully installed simply type in the command **docker** and check if it displays something like this (or similar)

```
barch584@PIbarch584:~ $ docker
Usage:  docker [OPTIONS] COMMAND

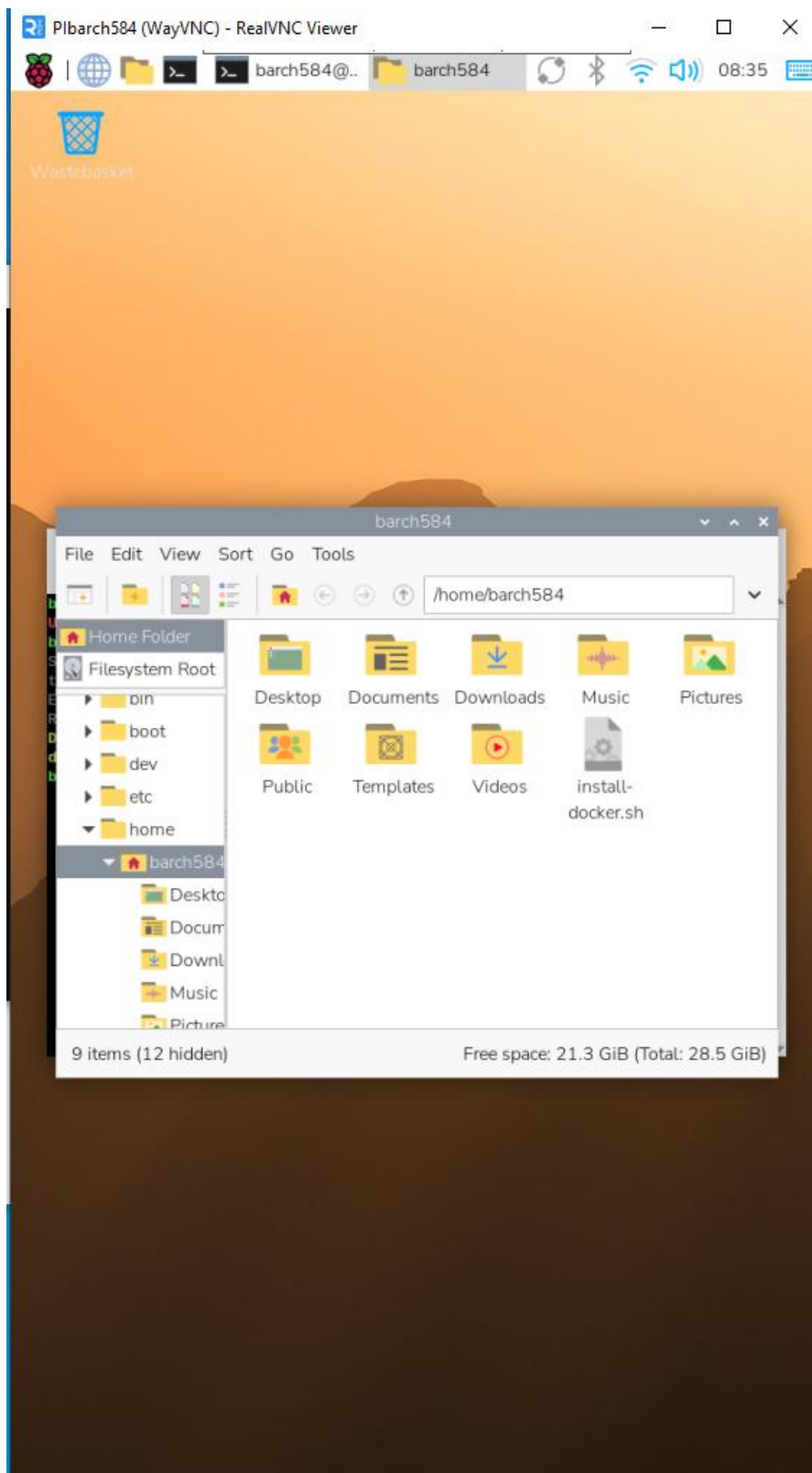
A self-sufficient runtime for containers

Common Commands:
  run         Create and run a new container from an image
  exec        Execute a command in a running container
  ps          List containers
  build        Build an image from a Dockerfile
  bake        Build from a file
  pull        Download an image from a registry
  push        Upload an image to a registry
  images      List images
  login        Authenticate to a registry
  logout      Log out from a registry
  search       Search Docker Hub for images
  version     Show the Docker version information
  info        Display system-wide information

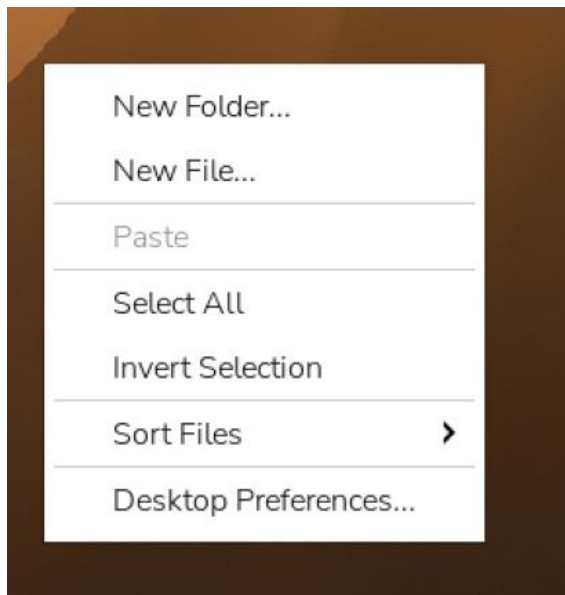
Management Commands:
  builder      Manage builds
  buildx*     Docker Buildx
  checkpoint   Manage checkpoints
  compose*    Docker Compose
  container    Manage containers
  context      Manage contexts
  image        Manage images
  manifest     Manage Docker image manifests and manifest lists
  model*      Docker Model Runner
  network      Manage networks
  plugin       Manage plugins
  system       Manage Docker
  volume       Manage volumes
```

Screen orientation

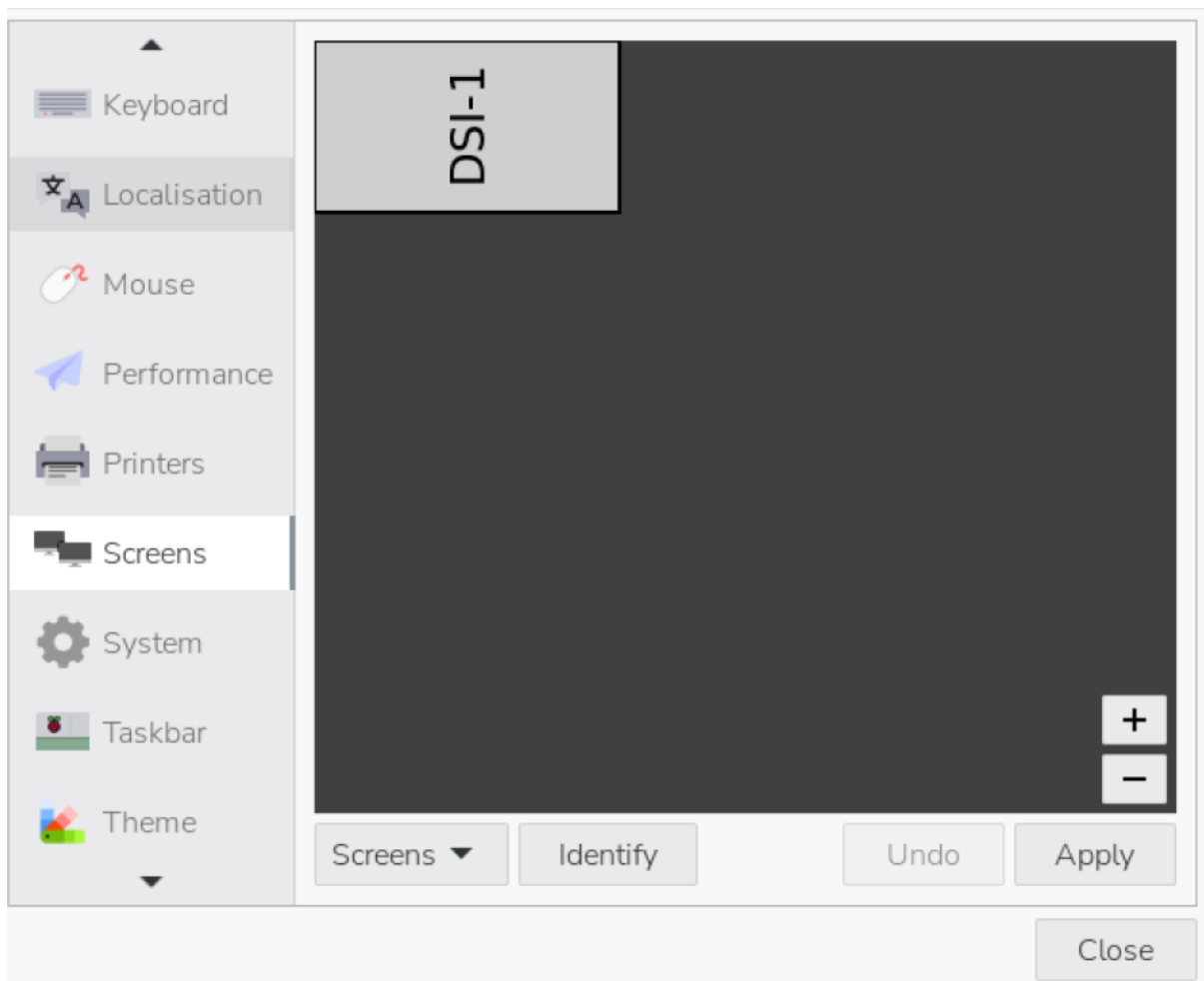
After connecting & receiving the display parts & PI together and connecting to VNC it should look something like this



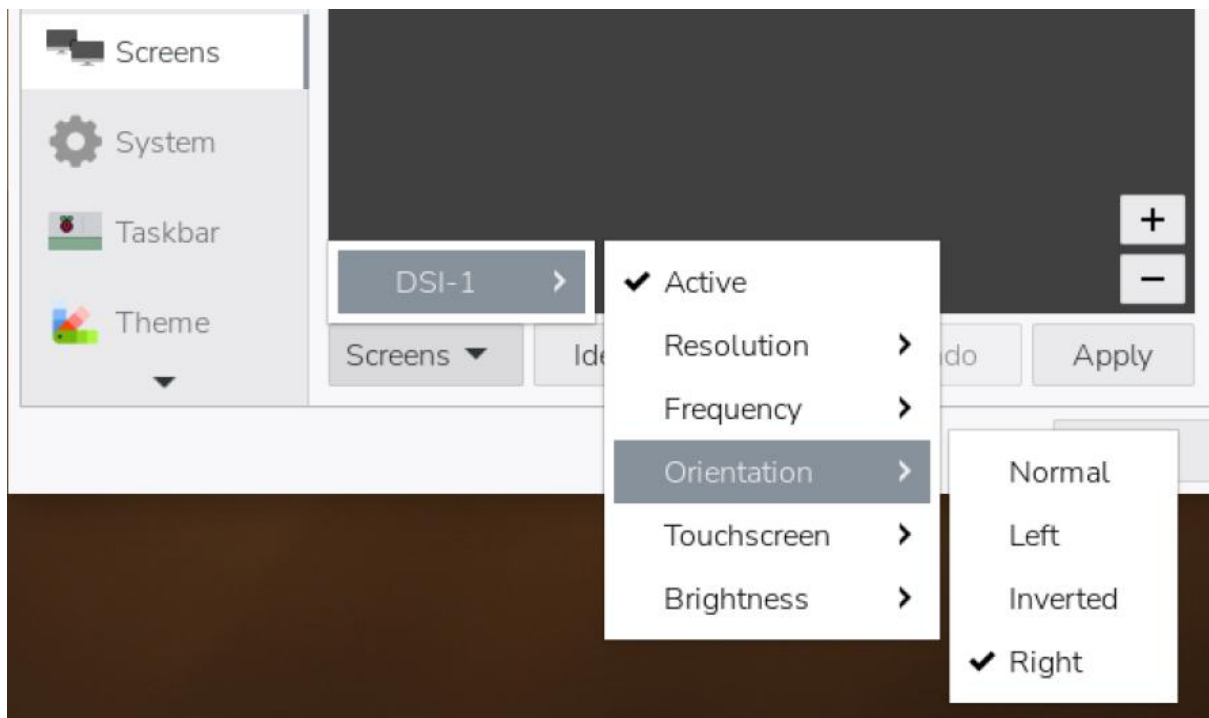
To change the orientation **right click on the desktop** > **Desktop Prefences...** >



Scroll down to **Screens**



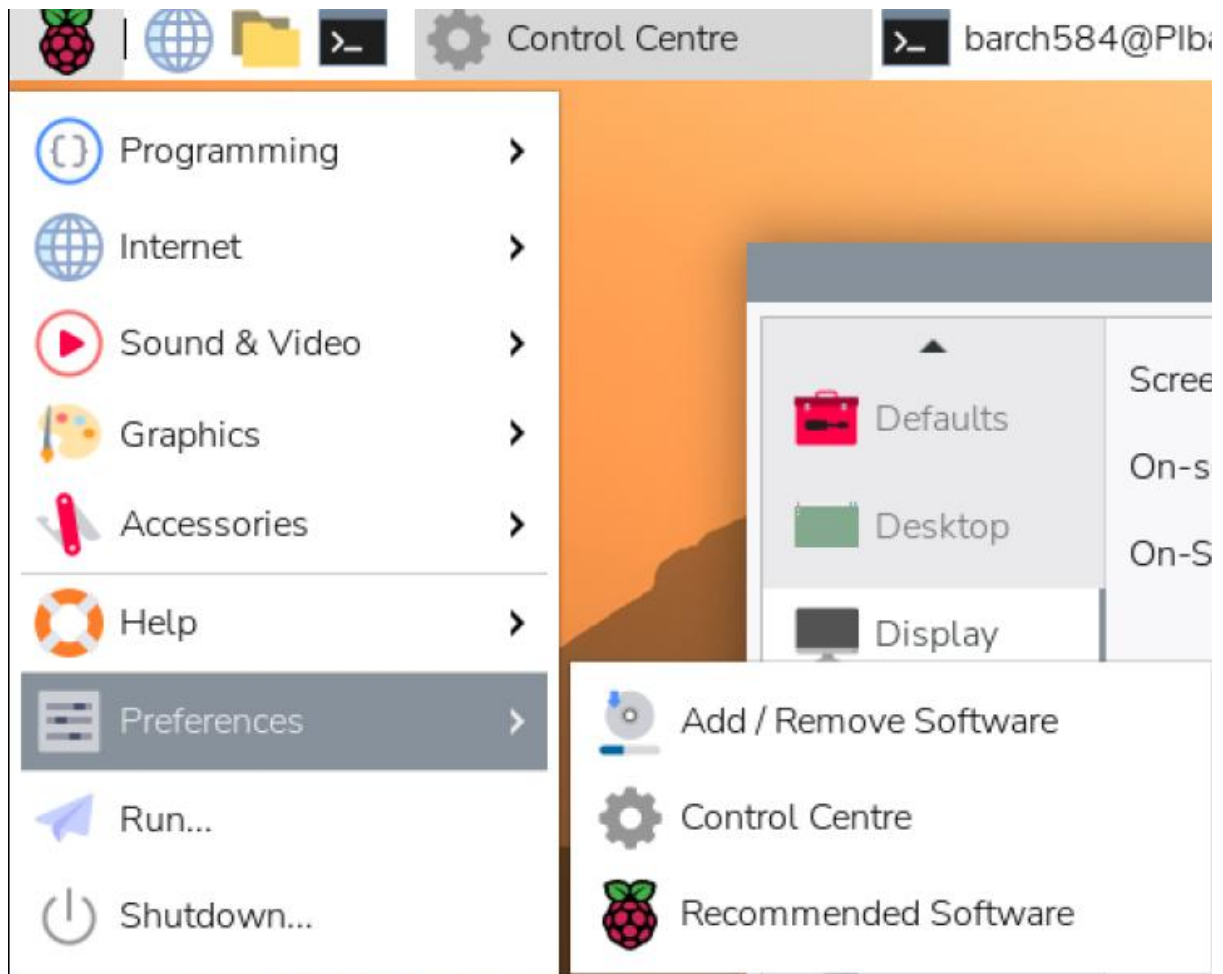
Click on **Screens button** > **DSI-1** > **Orientation** > choose **Right**



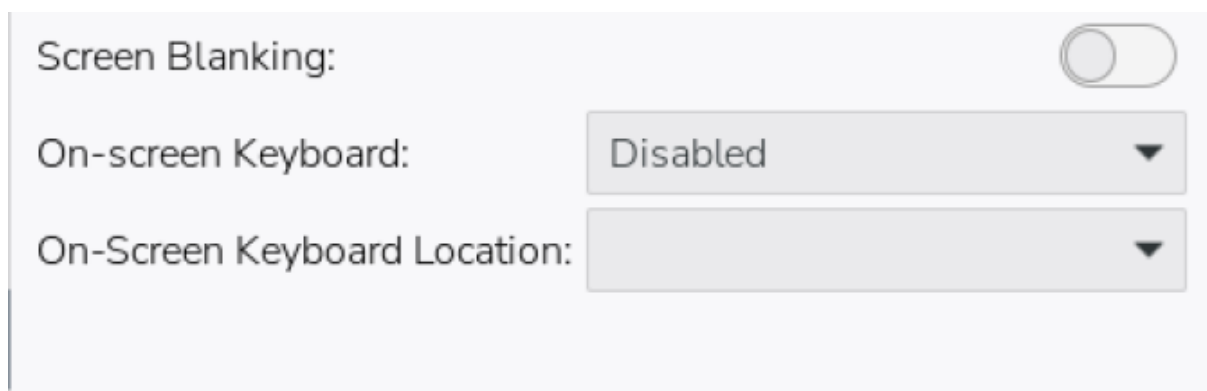
Reboot the machine & reopen VNC viewer

Disable on screen keyboard (optional)

Click on the **raspberry PI logo** > **Preferences** > **Control Centre** > **Display**



Change **On-screen Keyboard:** to **Disabled**



Create folder `/opt/station` & `/opt/station/www` & copy files

To create the 2 needed directories type in the termux:

cd /

sudo mkdir /opt/station

sudo mkdir /opt/station/www

```
barch584@PIbarch584:/opt $ sudo mkdir station
```

```
barch584@PIbarch584:/ $ sudo mkdir /opt/station/www
```

Now to move all firmware files:

**sudo mv /home/barch584/Desktop/Ressourcen/Firmware\ und\ Scripts/*
/opt/station/**

```
barch584@PIbarch584:/opt/station $ sudo mv /home/barch584/Desktop/Ressourcen/Fir  
mware\ und\ Scripts/* /opt/station/
```

Copy all the files in of the website folder in there:

**sudo mv /home/barch584/Desktop/Ressourcen/Station-Webseite/*
/opt/station/www**

```
barch584@PIbarch584:/opt/station $ sudo mv /home/barch584/Desktop/Ressourcen/Sta  
tion-Webseite/* /var/www/html
```

NOTE: Due to a incorrect information in the documentation I had to move all the website files in a newly created folder (/opt/station/www) from /var/www/html (so you can ignore the screenshot above) with the command:

sudo mv /var/www/html/* /opt/station/www

```
barch584@PIbarch584:/ $ sudo mv /var/www/html/* /opt/station/www
```

Create a docker container

With the following command you create a docker container & autostart with it unless the service is supposed to be disabled by the user:

```
docker run -d --name php --restart unless-stopped -p 8080:80 -v  
/tmp/station:/tmp/station -v /opt/station/www:/var/www/html php:8.4-apache
```

```
barch584@PIbarch584: / $ sudo docker run -d --name php --restart unless-stopped -p 8080:80 -v /tmp/station:/tmp/station -v /opt/station/www:/var/www/html php:8.4-apache  
Unable to find image 'php:8.4-apache' locally  
8.4-apache: Pulling from library/php  
4f4fb700ef54: Pull complete  
3fa20110e5dd: Pull complete  
d71c20bd3449: Pull complete  
b89cf3ec7a3e: Pull complete  
8caf2c340c63: Pull complete  
53138ea265d8: Pull complete  
def40a9103a7: Pull complete  
22003554cbf4: Pull complete  
b6a214c39dd4: Pull complete  
12ee6a10c3fa: Pull complete
```

Configure autostart

```
sudo apt -y install wtype
```

```
barch584@PIbarch584: ~/.config/labwc $ sudo apt -y install wtype  
Installing:  
wtype
```

```
sudo nano .config/labwc/autostart
```

Type in the document:

```
sudo docker run -d --name php --restart unless-stopped -p 8080:80 -v  
/tmp/station:/tmp/station -v /opt/station/www:/var/www/html php:8.4-apache
```

```
chromium http://localhost:8080 https://time.is/London --kiosk --noerrdialogs --  
disable-infobars --no-first-run --enable-features=OverlayScrollbar --start-  
maximized & ~/switchtab.sh
```

NOTE: not all of the script fitted in the screenshot

```
GNU nano 8.4 .config/labwc/autostart  
sudo docker run -d --name php --restart unless-stopped -p 8080:80 -v /tmp/station:/tmp/station -v /opt/station/www:/var  
chromium http://localhost:8080 https://time.is/London --kiosk --noerrdialogs --disable-infobars --no-first-run --enable-  
~/switchtab.sh
```

Next create and edit the file **switchtab.sh** in **~/switchtab.sh**:

Add the following:

```
#!/bin/bash

chromium_pid=$(pgrep chromium | head -1)
while [[ -z $chromium_pid ]]; do
    echo "Chromium browser is not running yet."
    sleep 5
    chromium_pid=$(pgrep chromium | head -1)
done
echo "Chromium browser process ID: $chromium_pid"
while true; do
    # Send Ctrl+Tab using `wtype` command
    wtype -M ctrl -P Tab -p Tab
    sleep 10
done
```

Source for the script above: <https://www.raspberrypi.com/tutorials/how-to-use-a-raspberry-pi-in-kiosk-mode/>

```
#!/bin/bash

chromium_pid=$(pgrep chromium | head -1)
while [[ -z $chromium_pid ]]; do
    echo "Chromium browser is not running yet."
    sleep 5
    chromium_pid=$(pgrep chromium | head -1)
done

echo "Chromium browser process ID: $chromium_pid"

while true; do
    wtype -M ctrl -P Tab -p Tab
    sleep 10
done
```

Next edit the permissions for the **switchtab.sh** that it can run without sudo

```
chmod +x ~/switchtab.sh
```

```
barch584@PIbarch584:~/.config/labwc $ sudo chmod +x ~/switchtab.sh
```

Now to set it in kiosk mode **simply reboot**

(OPTIONAL: FOR DEACTIVATING)

To be able for myself to take further configuration I commented the autostart files content with **sudo nano ~/.config/labwc/autostart** & adding hashtags at the beginning of both lines. Otherwise you could also access it remotely via a cmd with ssh, comment it there and reboot the PI

```
#chromium https://raspberrypi.com https://time.is/London --kiosk --noerrdialog
#~/switchtab.sh
```

Create folder **/tmp/station**

Already created by the docker command

Redirect sensor readings to result.csv

Create the result.csv file:

cd /tmp/station

sudo nano result.csv

NOTE: you may have to type something in the file for linux to be able to save it (the content does not matter)

```
barch584@PiBarch584:/ $ cd /tmp/station
barch584@PiBarch584:/tmp/station $ ls
barch584@PiBarch584:/tmp/station $ sudo nano result.csv
```

Redirect to that csv file by opening **sudo nano /opt/station/www/index.php**

Make sure the directory is set correctly to **/tmp/station/result.csv** like shown in the screenshot.

```
$csv = file("/tmp/station/result.csv");
if (!isset($csv)) {
```

MySQL setup

Mysql Database

The full database script is in **database.sql**. I created all 4 tables (Users, Stations, Collections & measurements)

Users table contains:

- Username (type: varchar & PK)
- First_name (type:varchar)
- Last_name (type:varchar)
- Adminflag (type: boolean)

Stations table contains

- serial_number (type: varchar & PK)
- name (type: varchar & PK)
- description (type: varchar & PK)

Measurements table contains

- id (type: int & auto_increment & primary key)
- timestamp (type: timestamp & default current_timestamp)
- temperature (type: int)
- humidity (type: int)
- air_pressure (type: int)
- light_intensity (type: int)
- air_quality (type: int)

Collections table contains

- name (type: varchar & primary key)
- description (type: varchar)

User_friends table contains

- user_id (type: int & PK)
- friend_id (type: int & PK)
- FK user_friends.user_id -> users.user_id
- FK user_friends.friend_id -> users.user_id

EDITS:

- Added **password**, **email**, and **role** to **Users** for authentication and authorization.
- Replaced **Adminflag** with role-based access control.
- Linked **Stations** to **Users** using a foreign key (station ownership).
- Increased measurement precision by changing values from **INT** to **DECIMAL**.
- Linked **Measurements** to **Stations** using a foreign key.
- Replaced collection name primary key with an auto-incremented ID.
- Linked **Collections** to their creating users.
- Added tables for:
 - measurement-collection relationships,
 - user access to collections,
 - friendship management with status tracking.
- Improved normalization and referential integrity across the schema.

```
DROP DATABASE IF EXISTS PIF_26;
CREATE DATABASE PIF_26;
USE PIF_26;

CREATE TABLE Users (
    username VARCHAR(50) PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    adminflag BOOLEAN
);

CREATE TABLE Stations (
    serial_number VARCHAR(50) PRIMARY KEY,
    name VARCHAR(50),
```

If copy pasted into phpMyAdmin editor & upon execution you should get something like this returned:

(NOTE: The local webserver used was Apache in this case in combination with phpMyAdmin in a **xampp** environment)

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0400 seconds.)

```
DROP DATABASE IF EXISTS PIF_26;
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0020 seconds.)

```
CREATE DATABASE PIF_26;
```

[Edit inline] [Edit] [Create PHP code]

⚠ Error: #1049 Unknown database 'PIF_26'

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0010 seconds.)

```
USE PIF_26;
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0160 seconds.)

```
CREATE TABLE Users ( username VARCHAR(50) PRIMARY KEY, first_name VARCHAR(50), last_name VARCHAR(50), email VARCHAR(100), adminflag BOOLEAN );
```

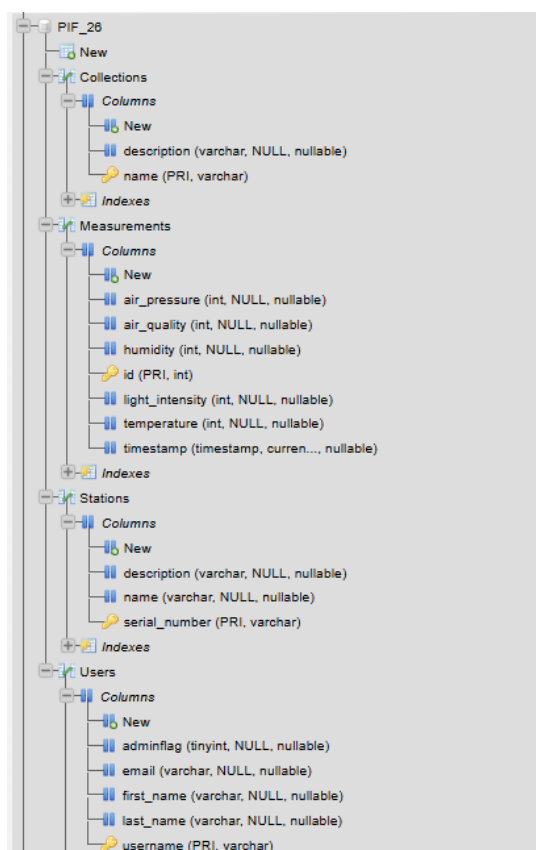
[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0150 seconds.)

```
CREATE TABLE Stations ( serial_number VARCHAR(50) PRIMARY KEY, name VARCHAR(50), description VARCHAR(200) );
```

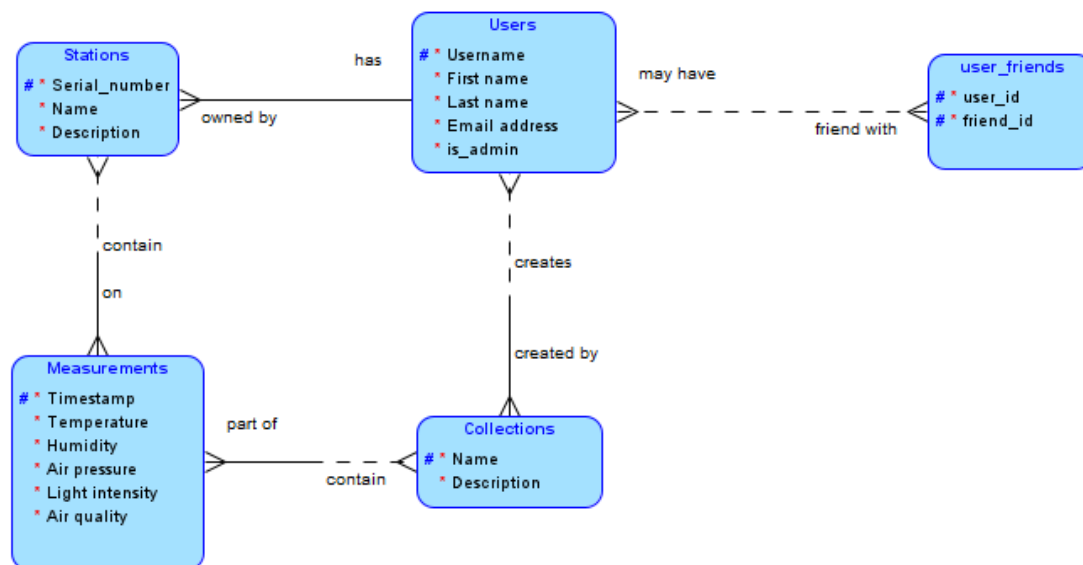
[Edit inline] [Edit] [Create PHP code]

On the left of **phpmyadmin** there should be a database with the name **PIF_26** to be seen with the corresponding tables as shown below:



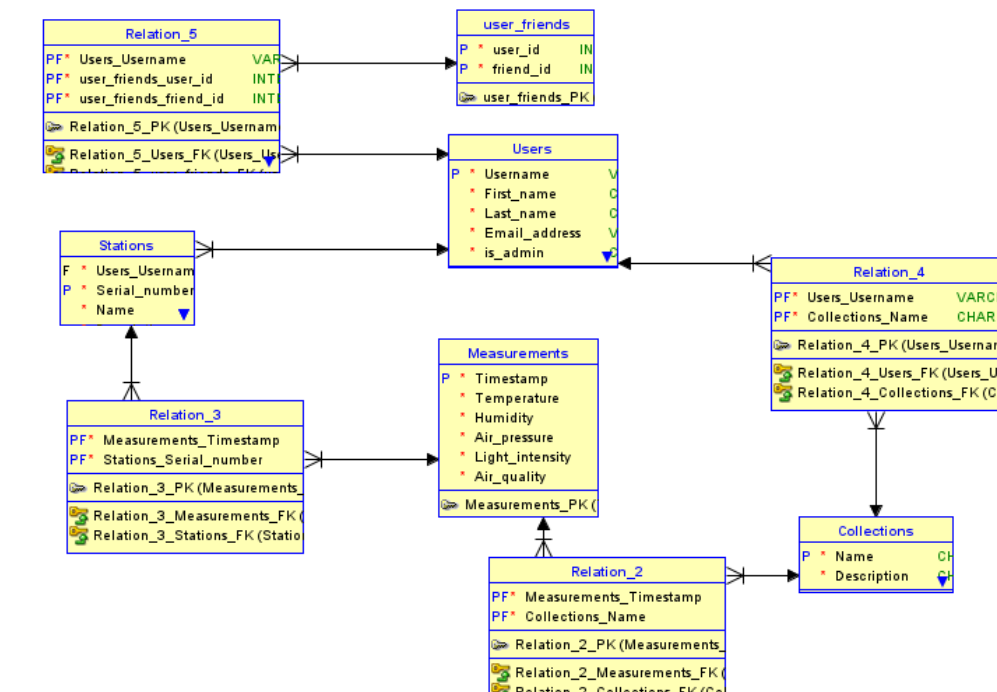
Conceptual data model

Here is the conceptual data model:



Logical data model

Here is the logical data model:



Webserver

Notes:

For this documentation I used the raspberryPi to install & configure everything mentioned in the user story.

Before starting everything it is a good idea to have everything up-to-date by running in the terminal **sudo apt update && sudo apt upgrade -y**

```
barch584@PIbarch584:~ $ sudo apt update && sudo apt upgrade -y
```

Apache installation

Install apache by typing **sudo apt install apache2**

```
barch584@PIbarch584:~ $ sudo apt install apache2
```

MariaDB installation

Install mariadb with **sudo apt install mariadb-server**

```
barch584@PIbarch584:~ $ sudo apt install mariadb-server
```

PHP installation

Install php with **sudo apt install php**

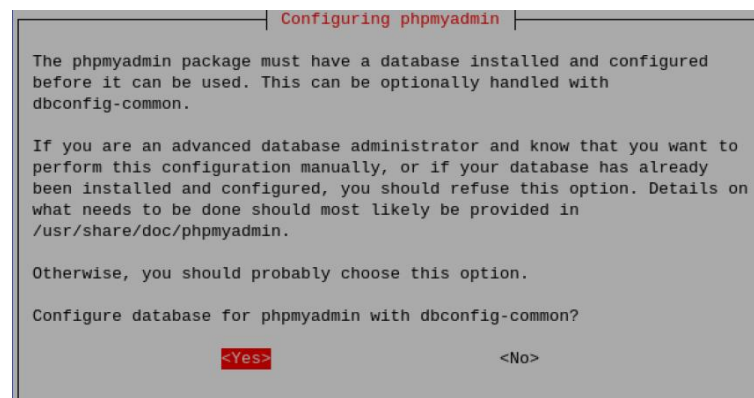
```
barch584@PIbarch584:~ $ sudo apt install php
```

phpMyAdmin installation

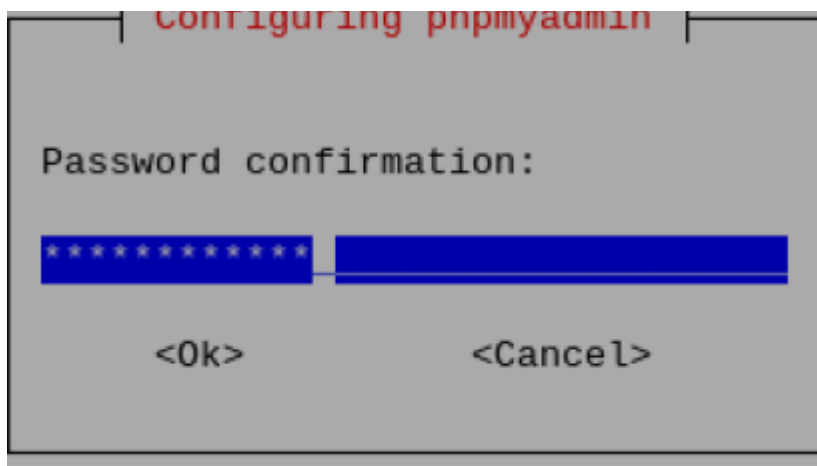
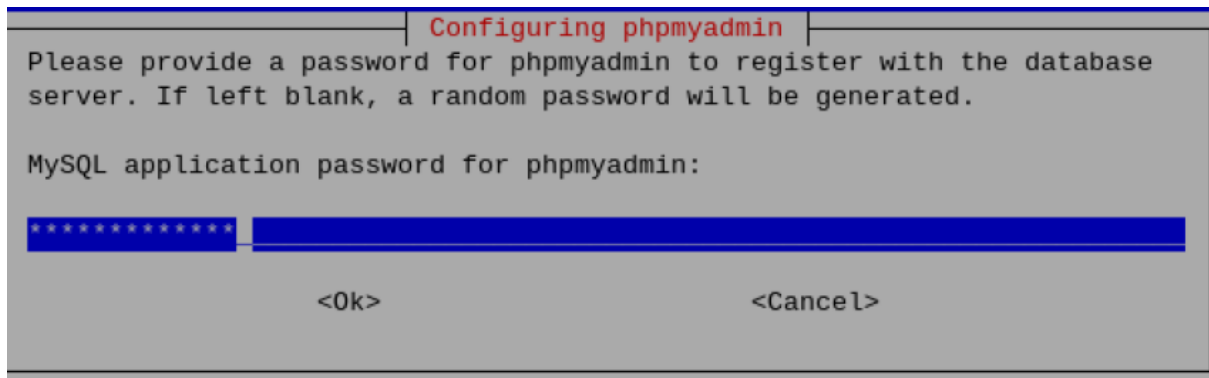
Install phpMyAdmin with **sudo apt install phpmyadmin**

```
barch584@PIbarch584:~ $ sudo apt install phpmyadmin
```

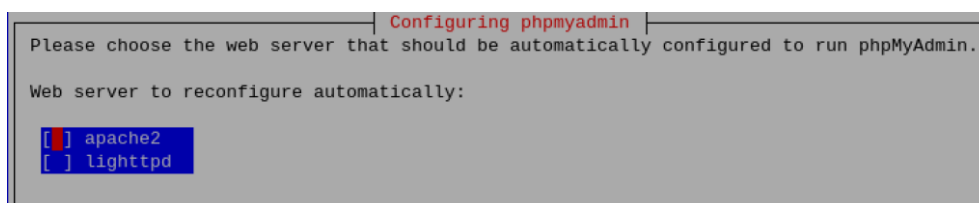
On this window select **Yes**



Then input your password for phpmyadmin to register (and confirm it).



During the installation select on this window **apache2**.

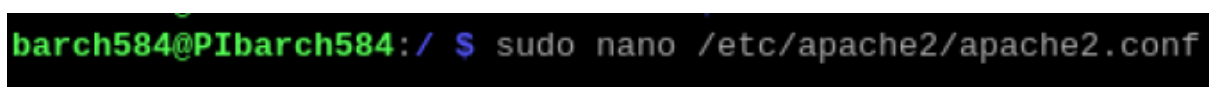


Afterwards the basic installation is completed.

phpMyAdmin configuration

Administrator account

You also need for this to work in a web browser to include it in the apache2.conf file. For that simply type in **sudo nano /etc/apache2/apache2.conf**



And include at the top of the file: **Include /etc/phpmyadmin/apache.conf**

```
include /etc/phpmyadmin/apache.conf
```

Now let's start by actually creating a user account to login into phpmyadmin, for which you start with **sudo mysql**.

```
barch584@PIbarch584:/$ sudo mysql
Welcome to the MariaDB monitor.  C
Your MariaDB connection id is 46
```

Now type in **CREATE USER '<yourusername>'@'localhost' IDENTIFIED BY '<yourpassword>';**

Example: **CREATE USER 'barch584'@'localhost' IDENTIFIED BY 'rootUSER123';**

And now with the user created give the user the needed permissions. (For this documentation to make it the easiest possible I will grant all permissions on the localhost to be an administrator account)

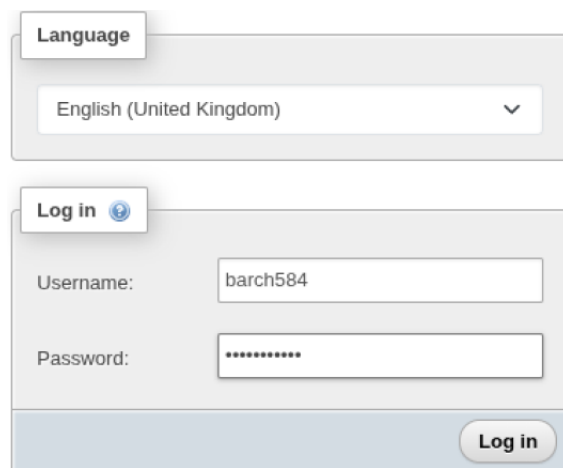
GRANT ALL PRIVILEGES ON *.* TO '<yourusername>'@'localhost' WITH GRANT OPTION;

Example: **GRANT ALL PRIVILEGES ON *.* TO 'barch584'@'localhost' WITH GRANT OPTION;**

```
MariaDB [(none)]> CREATE USER 'barch584'@'localhost' IDENTIFIED BY 'rootUSER123';
Query OK, 0 rows affected (0.005 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON *.* TO 'barch584'@'localhost' WITH GRANT OPTION;
Query OK, 0 rows affected (0.003 sec)
```

And by starting a browser and going to the link **localhost/phpmyadmin/** the following window should appear in the tab in which you simply type in your credentials created earlier in mysql:



Language

English (United Kingdom) ▼

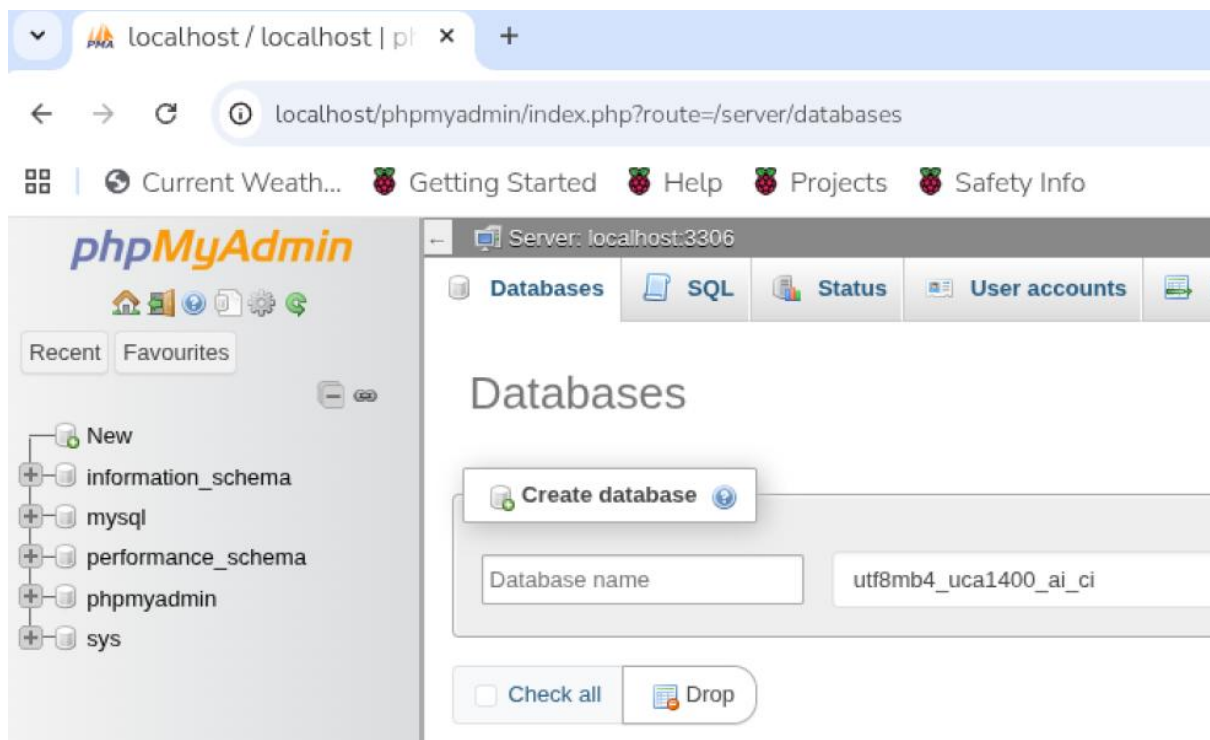
Log in ⓘ

Username: barch584

Password:

Log in

And afterwards your view looks something like this (or similar).



Restricted account

Create a new user account with a custom password which should be as secure as possible:

CREATE USER 'restricteduser'@'localhost' IDENTIFIED BY 'local1GUE\$t';

```
MariaDB [(none)]> CREATE USER 'restricteduser'@'localhost' IDENTIFIED BY 'local1GUE$t';
Query OK, 0 rows affected (0.006 sec)
```

And assign this restricteduser which is the end consumer account the SELECT & INSERT rights:

**GRANT SELECT,INSERT ON portableindoorfeedback.* TO
'restricteduser'@'localhost' WITH GRANT OPTION;**

```
MariaDB [(none)]> GRANT SELECT,INSERT ON portableindoorfeedback.*;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '' at line 1
MariaDB [(none)]> GRANT SELECT,INSERT ON portableindoorfeedback.* TO 'restricteduser'@'localhost' WITH GRANT OPTION;
Query OK, 0 rows affected (0.005 sec)
```

Install openssl and create self-signed SSL certificate

First install openssl with **sudo apt install openssl**

In my case it was already installed but this may vary.

```
barch584@PIbarch584:~ $ sudo apt install openssl
openssl is already the newest version (3.5.4-1~deb13u2+rpt1).
The following packages were automatically installed and are no longer required:
  libevent-pthreads-2.1-7t64 libwlroots-0.18
Use 'sudo apt autoremove' to remove them.
```

Before creating the key I created the appropriate folder for this:

```
sudo mkdir -p /etc/ssl/private
```

```
sudo chmod 700 /etc/ssl/private
```

```
barch584@PIbarch584:~ $ sudo mkdir -p /etc/ssl/private/
barch584@PIbarch584:~ $ sudo chmod 700 /etc/ssl/private/
```

Now generate the x509 certificate for the CA. With **sudo openssl req -new -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/apache-selfsigned.key -out /etc/ssl/certs/apache-selfsigned.crt**

```
barch584@PIbarch584:~ $ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/apache-selfsigned.key -out /etc/ssl/certs/apache-selfsigned.crt
-----
Country Name (2 letter code) [AU]:LU
State or Province Name (full name) [Some-State]:Luxembourg
Locality Name (eg, city) []:Luxembourg
Organization Name (eg, company) [Internet Widgits Pty Ltd]:REPIF_barch
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:PIbarch584.lu
Email Address []:barch584@school.lu
-----
```

In this case it would be for the prompts asked:

- LU
- Luxembourg
- Luxembourg
- REPIF_barch
- NOTHING HERE
- Pibarch584.lu OR <yourspecified hostname> OR <your IP> OR localhost
- barch584@school.lu OR <youremailaddress>

```
Country Name (2 letter code) [AU]:LU
State or Province Name (full name) [Some-State]:Luxembourg
Locality Name (eg, city) []:Luxembourg
Organization Name (eg, company) [Internet Widgits Pty Ltd]:REPIF_barch
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:PIbarch584.lu
Email Address []:barch584@school.lu
barch584@PIbarch584:~ $ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/apache-selfsigned.key -out /etc/ssl/certs/apache-selfsigned.crt
```

Make that only the root user can change the key, enable ssl & restart the server:

```
sudo chmod 600 /etc/ssl/private/apache-selfsigned.key
```

```
sudo a2enmod ssl
```

```
barch584@PIbarch584:~ $ sudo chmod 600 /etc/ssl/private/apache-selfsigned.key
barch584@PIbarch584:~ $ sudo a2enmod ssl
```

sudo systemctl restart apache2

```
barch584@PIbarch584:~$ sudo systemctl restart apache2
```

Now a new or even existing virtualhost file is needed to be (re)configured. For that, type in **cd /etc/apache2/sites-available/** and create a new file, I named mine **ssl-site.conf** with **sudo nano ssl-site.conf**

NOTE: In following screenshot I did it without the **cd**, but for the next commands I highly recommend to make it easier to navigate directly to the directory

```
barch584@admin:/$ sudo nano /etc/apache2/sites-available/ssl-site.conf
```

Now add the following text inside of it:

<VirtualHost *:443>

ServerName PIbarch584.lu

ServerAlias www.PIbarch584.lu

DocumentRoot /var/www/html

SSLEngine on

SSLCertificateFile /etc/ssl/certs/apache-selfsigned.crt

SSLCertificateKeyFile /etc/ssl/private/apache-selfsigned.key

</VirtualHost>

It simply enables that on port **443 (https)** use the self-signed certificate created earlier within the directory **/var/www/html** and use the normal log files.

Also the **Servername** & **ServerAlias** need to be changed depending on if you used an IP, localhost or a hostname in your certificate.

In my case I used a hostname named **christian.lu** created in earlier assignments and implemented it in there.

You simply modify hosts file with **sudo nano /etc/hosts** and add **<yourIP>** and the name from which it should resolve to that IP, like so:

```
127.0.0.1      PIbarch584.lu
192.168.4.137  PIbarch584.lu
```

192.168.4.137 Pibarch584.lu

127.0.0.1 Pibarch584.lu

Now save the file, restart the server and check in the status that the server is running correctly:

sudo systemctl restart apache2

sudo systemctl status apache2

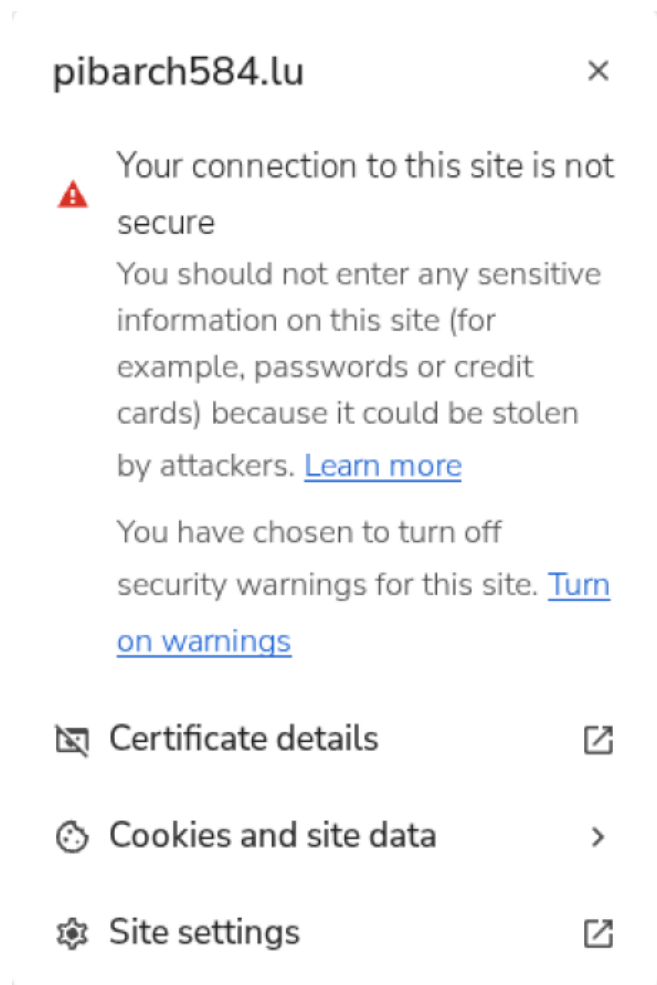
When typing the status command check that it displays some things in green like **enabled** and **active** like in the screenshot below:

```
root@PIbarch584:/etc/apache2/sites-available# sudo systemctl restart apache2
root@PIbarch584:/etc/apache2/sites-available# sudo systemctl status apache2
• apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-03-11 09:28:41 CET; 5s ago
 Invocation: 407a88d1bde6476b95e517a1edc0ddef
    Docs: https://httpd.apache.org/docs/2.4/
   Process: 3184 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
  Main PID: 3188 (apache2)
     Tasks: 7 (limit: 450)
        CPU: 305ms
   CGroup: /system.slice/apache2.service
           └─3188 /usr/sbin/apache2 -k start
             └─3190 /usr/sbin/apache2 -k start
               └─3191 /usr/sbin/apache2 -k start
                 └─3192 /usr/sbin/apache2 -k start
                   └─3193 /usr/sbin/apache2 -k start
                     └─3194 /usr/sbin/apache2 -k start
                       └─3195 /usr/sbin/apache2 -k start
```

You may also need to enable the new configuration file:

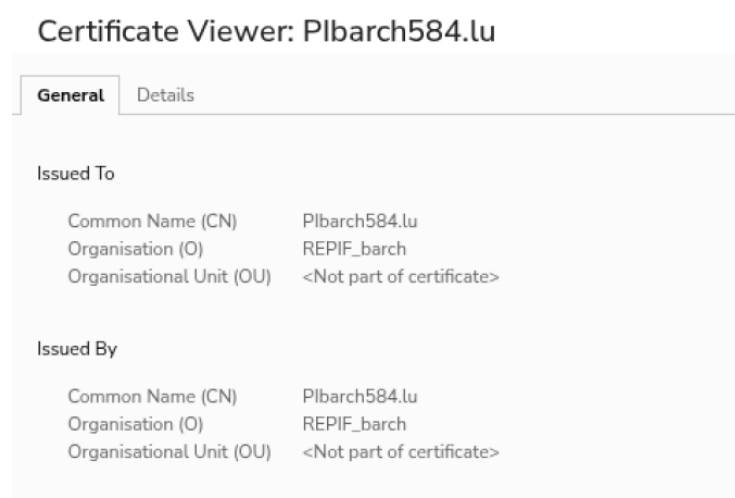
sudo a2ensite /etc/apache2/sites-available/ssl-site.conf

And by opening a web browser and going to that website, click on the **Not secure** button in the tab bar & click **Certificate Details**



Click on **Certificate details**

And there it should write the details of the certificate:



NOTE: for this all to work you must also complete the steps below in **Redirect HTTP traffic to HTTPS**

Redirect HTTP traffic to HTTPS

Start by modifying the 000-default.conf file with:

sudo nano /etc/apache2/sites-available/000-default.conf

```
barch584@admin:/etc/apache2/sites-available$ sudo nano /etc/apache2/sites-available/000-default.conf
```

Inside the **<VirtualHost *:80>** tags type in:

ServerName christian.lu

Redirect permanent / <https://christian.lu/>

Save the file and now all traffic is redirected from http -> https.

NOTE: again if you used localhost, IP or another hostname replace the link in **Redirect permanent** with whatever you configured earlier. In my case I choosed a DNS named **christian.lu** configured in earlier assignments.

Data transfer

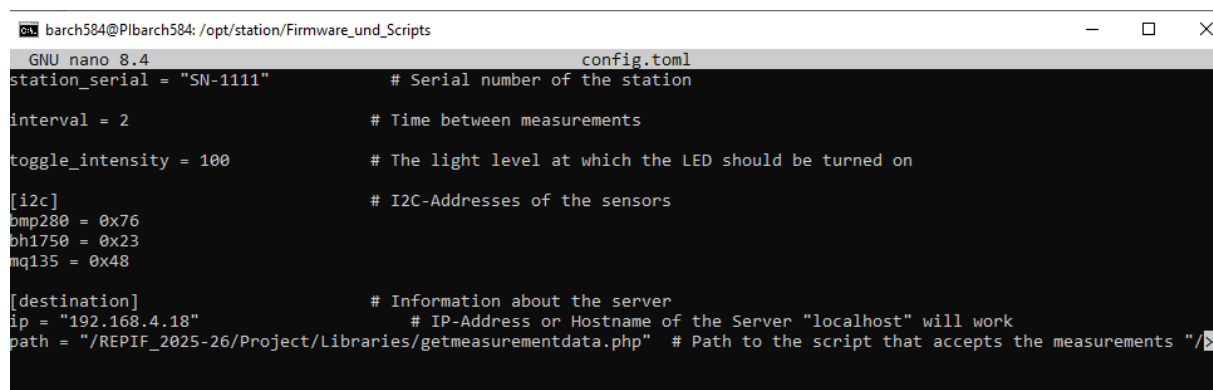
Config.toml

Firstly go to the **config.toml** file and start by changing the **station_serial** to a valid station registered in the database.

Change the IP to your host PC's IP. **192.168.4.18** was it in my case but this may vary. In addition to that write the path to the receiver file.

NOTE: There must be a / before the rest of the filepath

The result should look something like this:



```
GNU nano 8.4 config.toml
station_serial = "SN-1111"      # Serial number of the station
interval = 2                   # Time between measurements
toggle_intensity = 100         # The light level at which the LED should be turned on
[i2c]                          # I2C-Addresses of the sensors
bmp280 = 0x76
bh1750 = 0x23
mq135 = 0x48

[destination]                  # Information about the server
ip = "192.168.4.18"            # IP-Address or Hostname of the Server "localhost" will work
path = "/REPIF_2025-26/Project/Libraries/getmeasurementdata.php" # Path to the script that accepts the measurements
```

Next let's create the receiving file which is called (in this case) **getmeasurementdata.php**. It must be in the folder where you specified it in the **config.toml** file.

Receiver php script

The following script first gets the measurements via post request and simply inputs them into the database with an insert stmt. In bind_param you may have noticed that that **d** standing for **double** which simply means it expects numbers which could contain a comma and not entire numbers.

Now here is the content of the file:

```

<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Insert Measurement</title>

</head>

<body>

  <?php

    include("conndb.php");

    $temperature = $_POST['temperature'];

    $humidity  = $_POST['humidity'];

    $pressure  = $_POST['pressure'];

    $light     = $_POST['light'];

    $gas       = $_POST['gas'];

    $station   = $_POST['station_serial'];

    $timestamp = $_POST['timestamp'];

    $sqlInsert = $conn->prepare("INSERT INTO measurements (temperature, humidity,
pressure, light, gas, timestamp, fk_station_records) VALUES (?, ?, ?, ?, ?, ?, ?)");

    $sqlInsert->bind_param("dddddss", $temperature, $humidity, $pressure, $light, $gas,
$timestamp, $station);

    $sqlInsert->execute();

  ?>

</body>

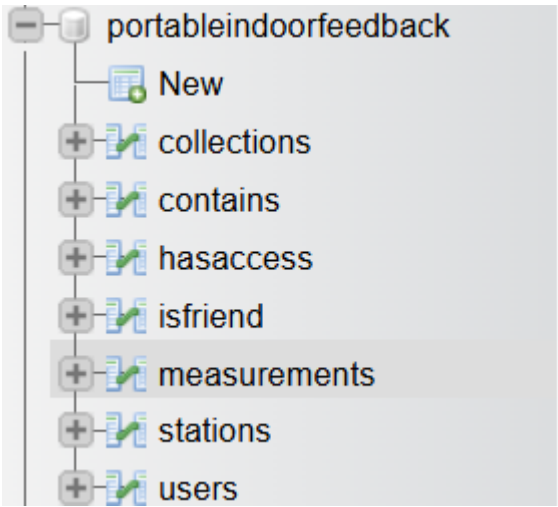
</html>

```

To correctly implement it, it must match the table names in your measurement table and also pay attention what you named your measurement table.

It also needs the correct connection data (to the database)

As you may notice mine is called **measurements** but change it if it is different.



Here is the structure of the table but this as well may be different.

pk_measurement	temperature	humidity	pressure	light	gas	timestamp	fk_station_records
----------------	-------------	----------	----------	-------	-----	-----------	--------------------

After confirming and having it implemented save it and check that the file location is correct as well as its filename.

Now after starting everything, which means having the raspberrypi on (with autostart activated with the proper content), turning on xampp (Apache & MySQL) it should return you results

	pk_measurement	temperature	humidity	pressure	light	gas	timestamp	fk_station_records
☐ Edit Copy Delete	653	26.84	50.00	996.35	0.00	1541.00	2026-03-19 10:47:39	SN-1111
☐ Edit Copy Delete	654	26.84	50.00	996.35	0.00	1543.00	2026-03-19 10:47:42	SN-1111
☐ Edit Copy Delete	655	26.84	50.00	996.37	0.83	1543.00	2026-03-19 10:47:44	SN-1111
☐ Edit Copy Delete	656	26.84	50.00	996.36	0.00	1543.00	2026-03-19 10:47:46	SN-1111
☐ Edit Copy Delete	657	26.84	50.00	996.36	0.00	1543.00	2026-03-19 10:47:48	SN-1111
☐ Edit Copy Delete	658	26.84	50.00	996.35	0.83	1544.00	2026-03-19 10:47:50	SN-1111
☐ Edit Copy Delete	659	26.84	50.00	996.36	0.00	1544.00	2026-03-19 10:47:52	SN-1111
☐ Edit Copy Delete	660	26.84	50.00	996.36	0.00	1544.00	2026-03-19 10:47:54	SN-1111
☐ Edit Copy Delete	661	26.85	50.00	996.37	0.00	1545.00	2026-03-19 10:47:56	SN-1111
☐ Edit Copy Delete	662	26.85	50.00	996.36	0.00	1544.00	2026-03-19 10:47:58	SN-1111
☐ Edit Copy Delete	663	26.85	50.00	996.36	0.83	1544.00	2026-03-19 10:48:00	SN-1111
☐ Edit Copy Delete	664	26.85	50.00	996.36	0.00	1544.00	2026-03-19 10:48:02	SN-1111
☐ Edit Copy Delete	665	26.85	50.00	996.36	0.00	1543.00	2026-03-19 10:48:04	SN-1111
☐ Edit Copy Delete	666	26.85	50.00	996.37	0.83	1543.00	2026-03-19 10:48:06	SN-1111

Autostart configuration

Here is the autostart configuration which is located in `~/.config/labwc/autostart`:

```
mkdir -p /tmp/station
```

```
python3 /opt/station/Firmware_und_Scripts/measure.py > /tmp/station/result.csv &
```

```
sudo systemctl start docker
```

```
docker start php
```

```
sleep 5
```

```
chromium http://localhost:8080 \
```

```
--kiosk --noerrdialogs --disable-infobars --no-first-run \
```

```
--enable-features=OverlayScrollbar --start-maximized &
```

```
Ctrl barch584@Plbarch584: /opt/station/Firmware_und_Scripts
GNU nano 8.4 /home/barch584/.config/labwc/autostart
mkdir -p /tmp/station
python3 /opt/station/Firmware_und_Scripts/measure.py > /tmp/station/result.csv &
sudo systemctl start docker
docker start php

sleep 5

chromium http://localhost:8080 \
--kiosk --noerrdialogs --disable-infobars --no-first-run \
--enable-features=OverlayScrollbar --start-maximized &
```

Measurements file

Now for the form which can import test data, here is my design of my form but this of course may vary. This is in my measurements.php file and looks like this:

Measurements

Dinosaurier

123

123

123

123

213

Create test Measurement

And the data gets later shown on the website

Time	Temperature	Humidity	Pressure	Light	Gas
2026-03-19 15:09:03	123 °C	123 %	123 hPa	123 lx	213 ppm
2026-03-19 15:09:03	123 °C	123 %	123 hPa	123 lx	213 ppm
2026-03-19 15:09:02	123 °C	123 %	123 hPa	123 lx	213 ppm
2026-03-19 15:09:02	123 °C	123 %	123 hPa	123 lx	213 ppm
2026-03-19 15:09:02	123 °C	123 %	123 hPa	123 lx	213 ppm

And the db:

☐	Edit	Copy	Delete	2066	123.00	123.00	123.00	123.00	213.00	2026-03-19 15:09:02	SN-1111
☐	Edit	Copy	Delete	2067	123.00	123.00	123.00	123.00	213.00	2026-03-19 15:09:02	SN-1111
☐	Edit	Copy	Delete	2068	123.00	123.00	123.00	123.00	213.00	2026-03-19 15:09:02	SN-1111
☐	Edit	Copy	Delete	2069	123.00	123.00	123.00	123.00	213.00	2026-03-19 15:09:03	SN-1111
☐	Edit	Copy	Delete	2070	123.00	123.00	123.00	123.00	213.00	2026-03-19 15:09:03	SN-1111

The following script allows to insert data into the database. Simply by making a \$.post request, give it the data and insert it. By updating the table you instantly get the new value. The filter is still experimentative and not working yet and the clearfilter simply resets the value of the table & the input fields where you could filter by dates & times.

Below is the current measurements.php script for the form:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Measurements</title>
```

```
<link rel="stylesheet" href="../Styles/styles.css">
```

```
</head>
```

```
<body class="light-theme">
```

```
<script src="../Libraries/JS/jquery-3.7.1.min.js"></script>
```

```
<?php
```

```
include("../Libraries/navbar.php");
```

```
include("../Libraries/conndb.php");
```

```
include("../Libraries/createmasurement.php");
```

```
?>
```

```
<?= createnavbar("Measurements"); ?>
```

```
<h1>Measurements</h1>
```

```
<?php
```

```
$selectstationsstmt = $conn->prepare("SELECT * FROM stations WHERE  
fk_user_owns = ?");
```

```
$selectstationsstmt->bind_param("i", $_SESSION['username']);
```

```
$selectstationsstmt->execute();
```

```
$result = $selectstationsstmt->get_result();
```

```
?>
```

```
<div class="form-card">
```

```
<div class="form-group">
```

```
<select name="stationslist" id="stationslist">
```



```

<?php
    while ($row = $result->fetch_assoc()) {
        echo "<option value='" . $row['pk_serialNumber'] . "'>" .
htmlspecialchars($row['name']) . "</option>";
    }
?>

</select><br>

<input type="text" placeholder="Temperature (in °C)"><br>
<input type="text" placeholder="Humidity (in %)"><br>
<input type="text" placeholder="Pressure (in hPa)"><br>
<input type="text" placeholder="Light (in lx)"><br>
<input type="text" placeholder="Gas (in kOhm)"><br>
<button type="submit" value="Create measurement"
id="createMeasurementButton">Create test Measurement</button>

</div>

</div>

<div class="form-card">
    <div class="form-group">
        <label>From</label>

        <input type="datetime-local" id="fromTime">
    </div>

    <div class="form-group">
        <label>To</label>

        <input type="datetime-local" id="toTime">
    </div>

    <button id="filterTime">Filter</button>

```

```
<button id="clearFilter">Clear</button>
```

```
</div>
```

```
<table id="tablemeasurements">
```

```
<tr>
```

```
<th data-col="0">Time <span class="arrow"></span></th>
```

```
<th data-col="1">Temperature <span class="arrow"></span></th>
```

```
<th data-col="2">Humidity <span class="arrow"></span></th>
```

```
<th data-col="3">Pressure <span class="arrow"></span></th>
```

```
<th data-col="4">Light <span class="arrow"></span></th>
```

```
<th data-col="5">Gas <span class="arrow"></span></th>
```

```
</tr>
```

```
</table>
```

```
<script>
```

```
let sortDirections = {};
```

```
let activeColumn = null;
```

```
$("#createMeasurementButton").click(start);
```

```
// ----- LOAD DATA -----
```

```
function start() {
```

```
$.post("../Libraries/createmeasurement.php", {
```

```
stationslist: $("#stationslist").val(),
```

```
temperature: $("input[placeholder='Temperature (in °C)']").val(),
```

```
humidity: $("input[placeholder='Humidity (in %)']").val(),
```

```
pressure: $("input[placeholder='Pressure (in hPa)']").val(),
```

```
light: $("input[placeholder='Light (in lx)']").val(),
```

```
gas: $("input[placeholder='Gas (in kOhm)']").val()
```

```
}, function(data) {
```

```

        $("#tablemeasurements tr:first").after(data);

        applyCurrentSort();

    });

}

function loadMeasurements() {

    $.post("../Libraries/showmeasurements.php", {

        station: $("#stationslist").val()

    }, function(data) {

        $("#tablemeasurements").append(data);

    });

}

loadMeasurements();

// ----- TIMESTAMP FILTER -----

$("#filterTime").click(function() {

    let from = ($("#fromTime").val() ? new Date($("#fromTime").val()) : null;

    let to = ($("#toTime").val() ? new Date($("#toTime").val()) : null;


    $("#tablemeasurements tr:gt(0)").each(function() {

        let timeText = $(this).children("td").eq(0).text().trim();

        let rowTime = new Date(timeText);

        let show = true;

        if (from && rowTime < from) show = false;

        if (to && rowTime > to) show = false;


        $(this).toggle(show);

    });

});

$("#clearFilter").click(function() {

```

```
$("#fromTime, #toTime").val("");
```

```
$("#tablemeasurements tr").show();
```

```
});
```

```
</script>
```

```
</body>
```

```
</html>
```

Backup server

Allow external MySQL database access & restricted account

Let's start with the 50-server.cnf config file to allow external connections:

Sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf

```
barch584@PIbarch584:/etc/mysql/conf.d $ sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

Change the **bind-address** to **0.0.0.0**

```
# Instead of skip-networking the default is now to listen on *
# localhost which is more compatible and is what will be used
bind-address            = 0.0.0.0
```

Next create the user with a password with this command. It authorizes the user to have as demanded full access to the entire **portableindoorfeedback** db. Click on **Go** to execute it:

CREATE USER 'pifuser26'@'%' IDENTIFIED BY 'PIFu\$er123';

GRANT ALL PRIVILEGES ON portableindoorfeedback.* TO 'pifuser26'@'%';

FLUSH PRIVILEGES;

CREATE USER 'pifuser26'@'host.docker.internal' IDENTIFIED BY 'PIFu\$er123';

**GRANT ALL PRIVILEGES ON portableindoorfeedback.* TO
'pifuser26'@'host.docker.internal';FLUSH PRIVILEGES;**

```
MariaDB [(none)]> CREATE USER 'pifusr26'@'%' IDENTIFIED BY 'PIFu$er123';
Query OK, 0 rows affected (0.012 sec)

MariaDB [(none)]> DROP USER 'pifusr26'@'%';
Query OK, 0 rows affected (0.003 sec)

MariaDB [(none)]> CREATE USER 'pifuser26'@'%' IDENTIFIED BY 'PIFu$er123';
Query OK, 0 rows affected (0.005 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON portableindoorfeedback.* to 'pifuser26'@'%';
Query OK, 0 rows affected (0.010 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.003 sec)

MariaDB [(none)]> CREATE USER 'pifuser26'@'host.docker.internal' IDENTIFIED BY 'PIFu$er123';
Query OK, 0 rows affected (0.003 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON portableindoorfeedback.* TO 'pifuser26'@'host.docker.internal';
Query OK, 0 rows affected (0.003 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> exit
Bye
```

Create a new virtual machine

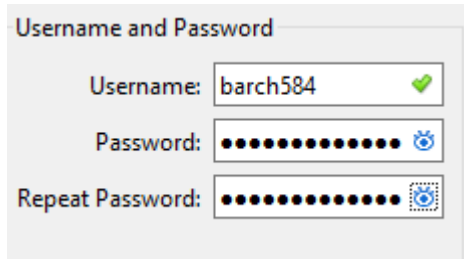
For that I created a new VM:

- ➔ RAM: 12GiB
- ➔ CPU cores: 4
- ➔ Storage: 50GB
- ➔ OS: Ubuntu Server

And a username & password are needed as well:

Username: <yourIAM>

Password: <yourstrongpassword>



Upon startup configure your keyboard layout & update everything:

sudo dpkg-reconfigure keyboard-configuration

```
barch584@PIF2526:~$ sudo dpkg-reconfigure keyboard-configuration
```

sudo apt update && sudo apt upgrade -y

```
barch584@PIF2526:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for barch584:
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:1 https://lu.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 https://lu.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 https://lu.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
1 package can be upgraded. Run 'apt list --upgradable' to see it.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following upgrades have been deferred due to phasing:
  ubuntu-drivers-common
0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.
barch584@PIF2526:~$
```

And of course, do not forget to reboot to apply all changes:

sudo reboot

```
barch584@PIF2526:~$ sudo reboot
```

Also, it is a good idea to install openssh-server with:

sudo apt install openssh-server

```
barch584@PIF2526:~$ sudo apt install openssh-server
```

Export database setup

The first step is to create a new folder wherever you want, but it needs writing permissions. (For testing purposes I will give everyone writing permissions):

Sudo mkdir ~/mysqlbackups

Sudo chmod 777 ~/mysqlbackups

```
barch584@PIF2526:~$ sudo mkdir ~/mysqlbackups
```

```
barch584@PIF2526:~$ sudo chmod 777 ~/mysqlbackups/
```

I also would highly recommend to create a **.my.cnf** file to not have to type a password each time. For that, create a new file in your home directory:

cd ~

sudo nano .my.cnf

```
barch584@PIF2526:~$ sudo nano .my.cnf
```

And inside of it, type in the following:

[client]

User=<yourdbusername>

Password=<yourdbuserpassword>

Host=<ipofyoursqlserver>

```
GNU nano 7.2
[client]
user=pifuser26
password=PIFu$er123
host=192.168.4.18
```

Now simply create a first test backup:

mysqldump --column-statistics=0 portableindoorfeedback >
~/mysqlbackups/db.sql

```
barch584@PIF2526:~/mysqlbackups$ mysqldump --column-statistics=0 portableindoorfeedback > ~/mysqlbackups/db.sql
```

And the file gets created afterwards.

```
barch584@PIF2526:~/mysqlbackups$ ls  
db.sql
```

Crontab

Let's automate it by applying the command with a crontab and giving it better more distinguishing names:

```
0 8 * * * mysqldump --column-statistics=0 portableindoorfeedback >  
$HOME/mysqlbackups/db_$(date +%Y-%m-%d_%H-%M-%S).sql
```


Workshop

PVC plates assembly

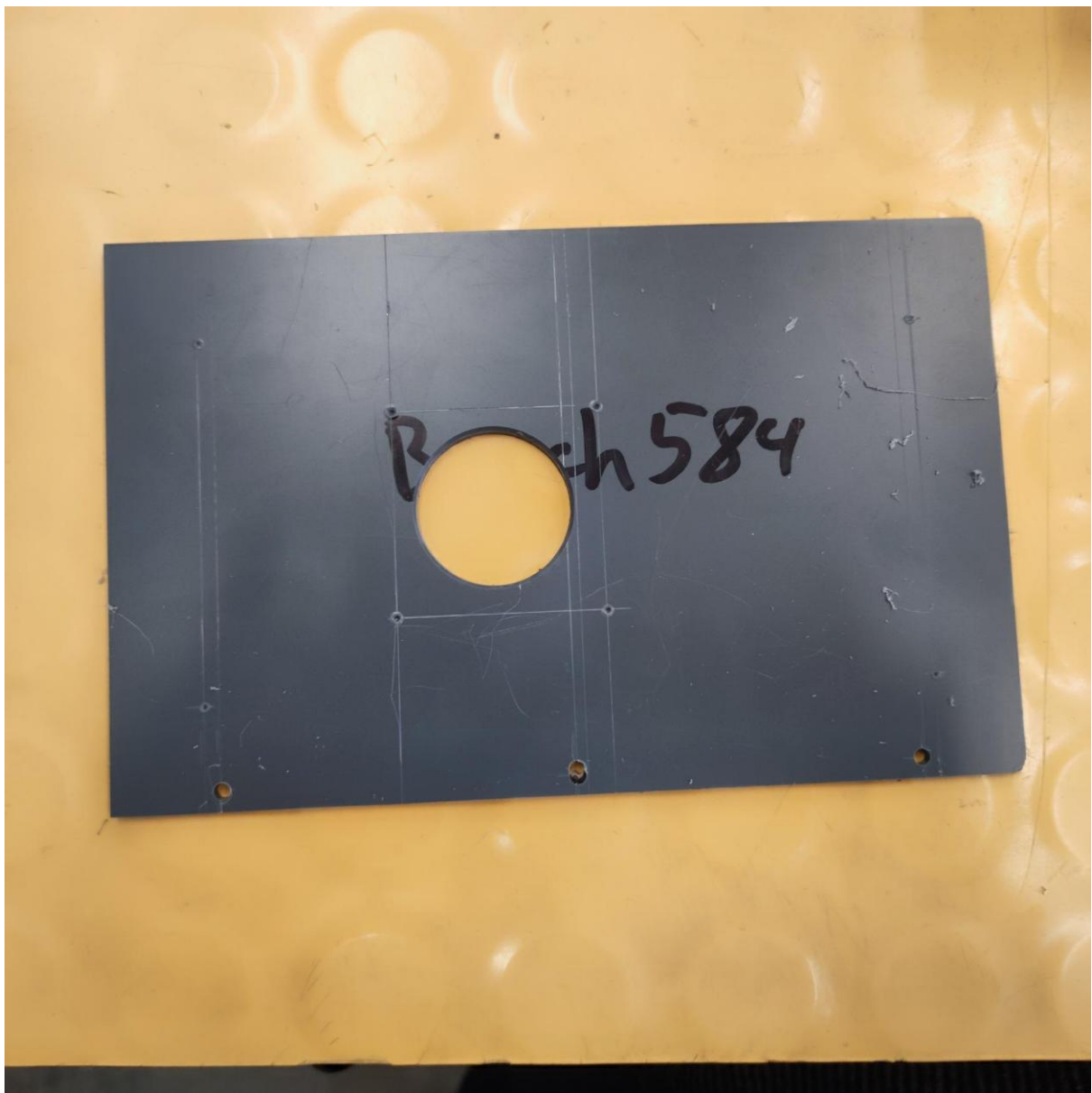
Display plate

This is the starter display plate

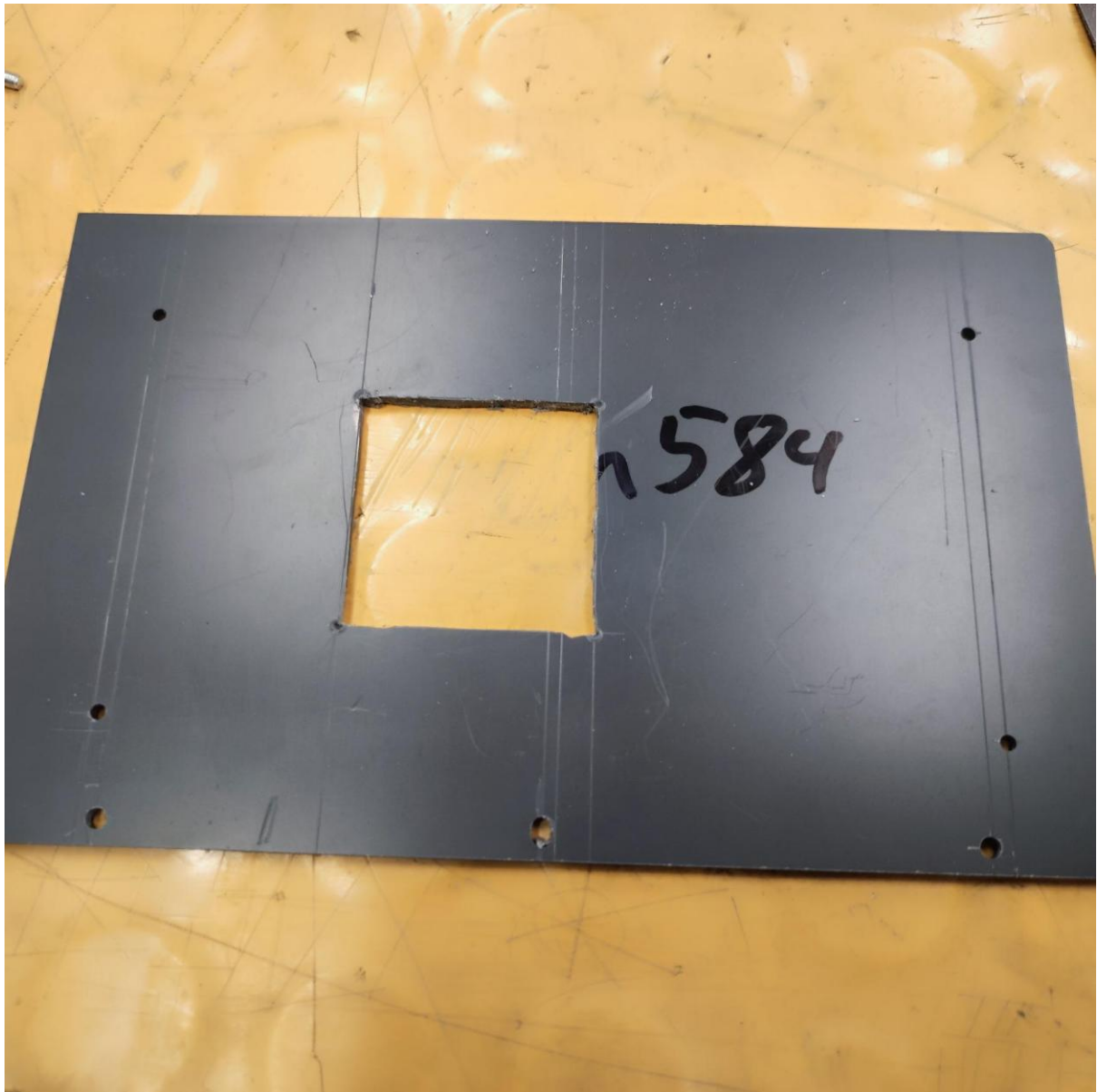


The display plate got in the first step everything cut into it with a scribing block according to the schematic to later have it center punched. This made sure that every drilling was smooth and more precise.

The first holes drilled into were the 3 holes (3mm) to connect the display & base plate together later. Next for the big “square” more to the center of the plate I used a step bit until the maximum size to open it up to some extend already.

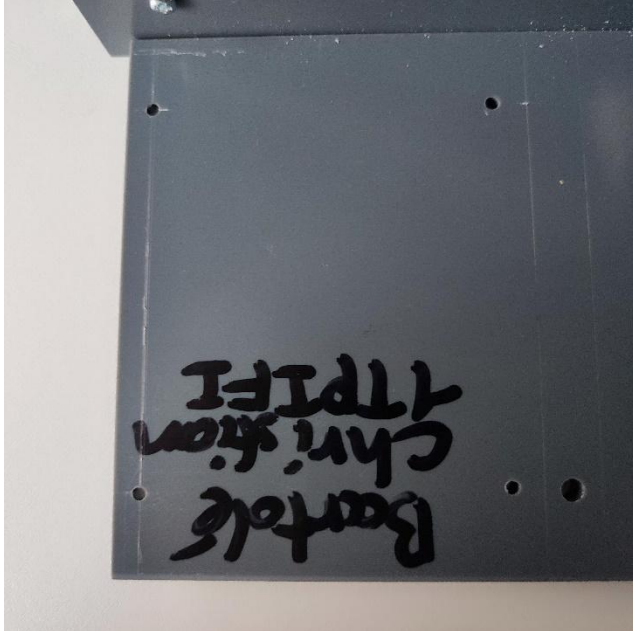


Afterwards I filed the previous created circle into an actual square with a dimension of 40*40mm. Quickly followed by drilling all remaining 2.5mm holes remaining according to the schematic to later attach a display to it.



The base plate:

Like the previous plate I cut into it with a scribing block according to the schematic to later have it center punched. Next, I drilled the holes for the custom PCB as well as for the raspberrypi



Here how the raspberrypi is connected to the plate. In addition to the 4 screws it also has for each a washer, a nut & a 5mm space sleeve between the PI & the plate.:

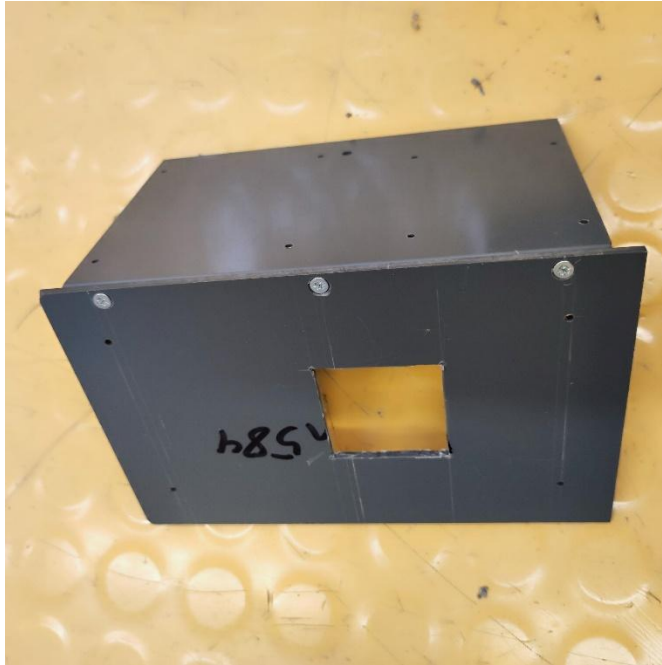
(NOTE: Due to 1-2mm difference between the schematic and the real plate, the screws for screwing the raspberrypi had to be screwed into the plate a bit sideways.)



Before connecting both plates the base plate needed 3 holes on the top part of the plate drilled with 2.3mm & an M3 drill afterwards for the screws.

Plate assembly

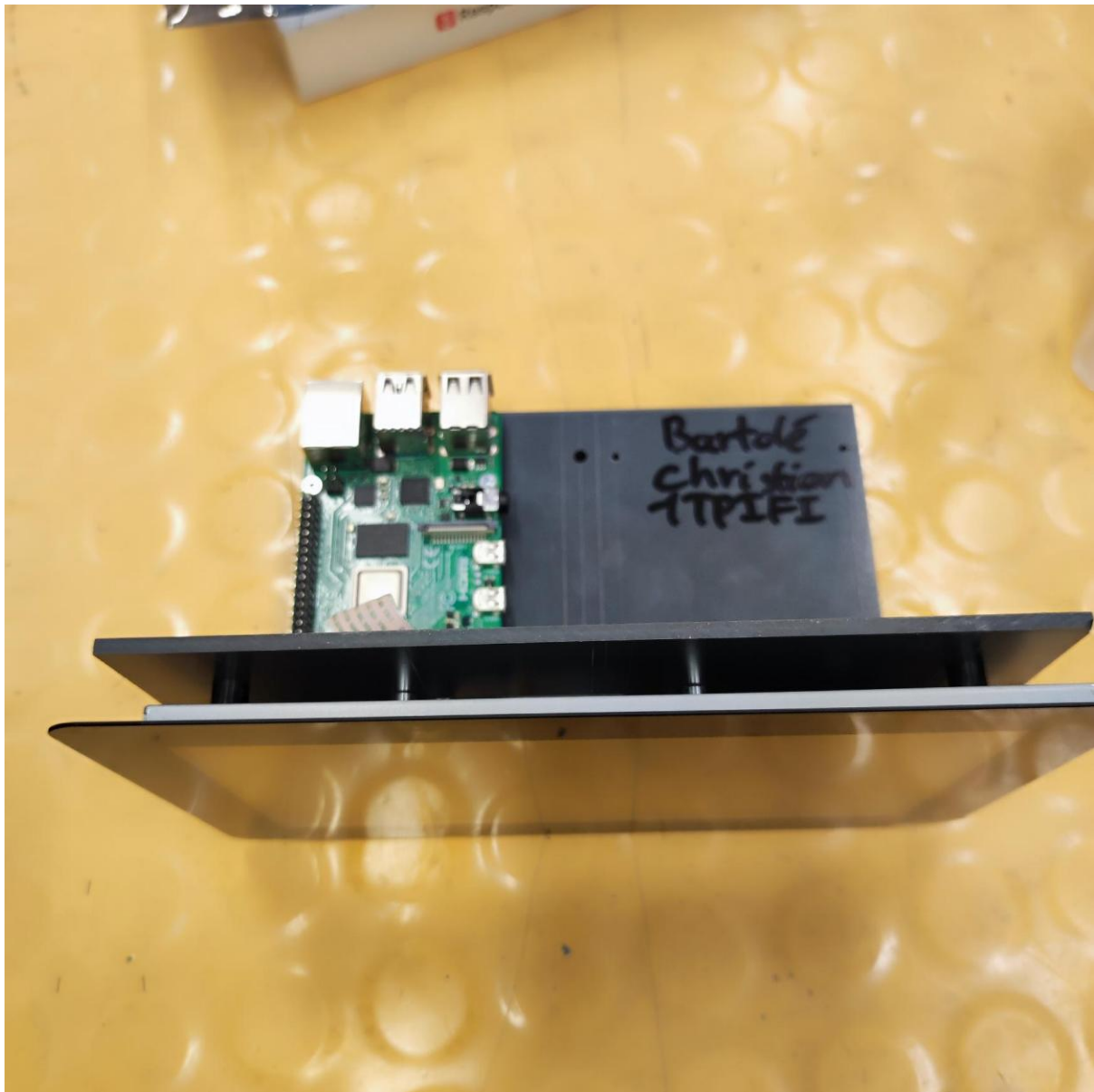
Now that both plates are prepared, connect both together with M3 screws provided.



And now it looks something like this:



The next thing put onto was the display with the 4 screws earlier created more to the corner of the display plate like so with the displays threaded holes with some 5mm black spacers in between:

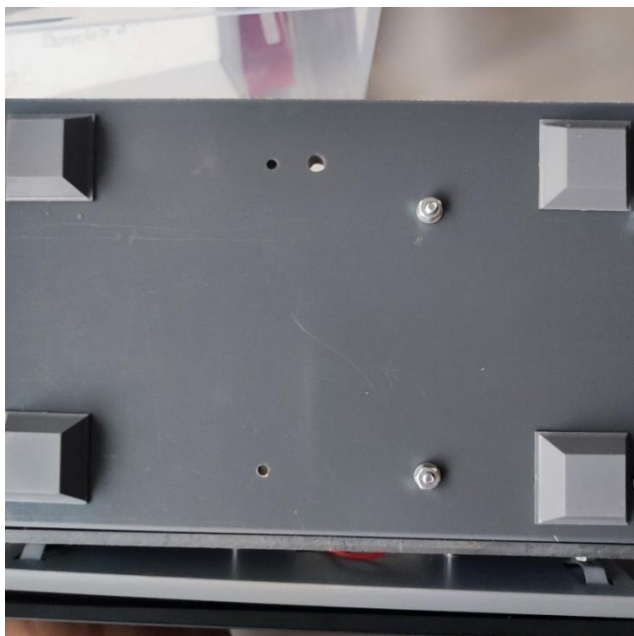


NOTE: due to a difference of 1-2mm 1 screw did not go into the threaded insert and had to be left unconnected by attaching the screw inside with a spacer and a nut.

Now the necessary cables (DISPLAY & power cable for the display are connected to the raspberrypi). The red pin on a 5V pin & the black on a GND pin on the PI and the cable connected to the JST connector labeled **DISPLAY**)



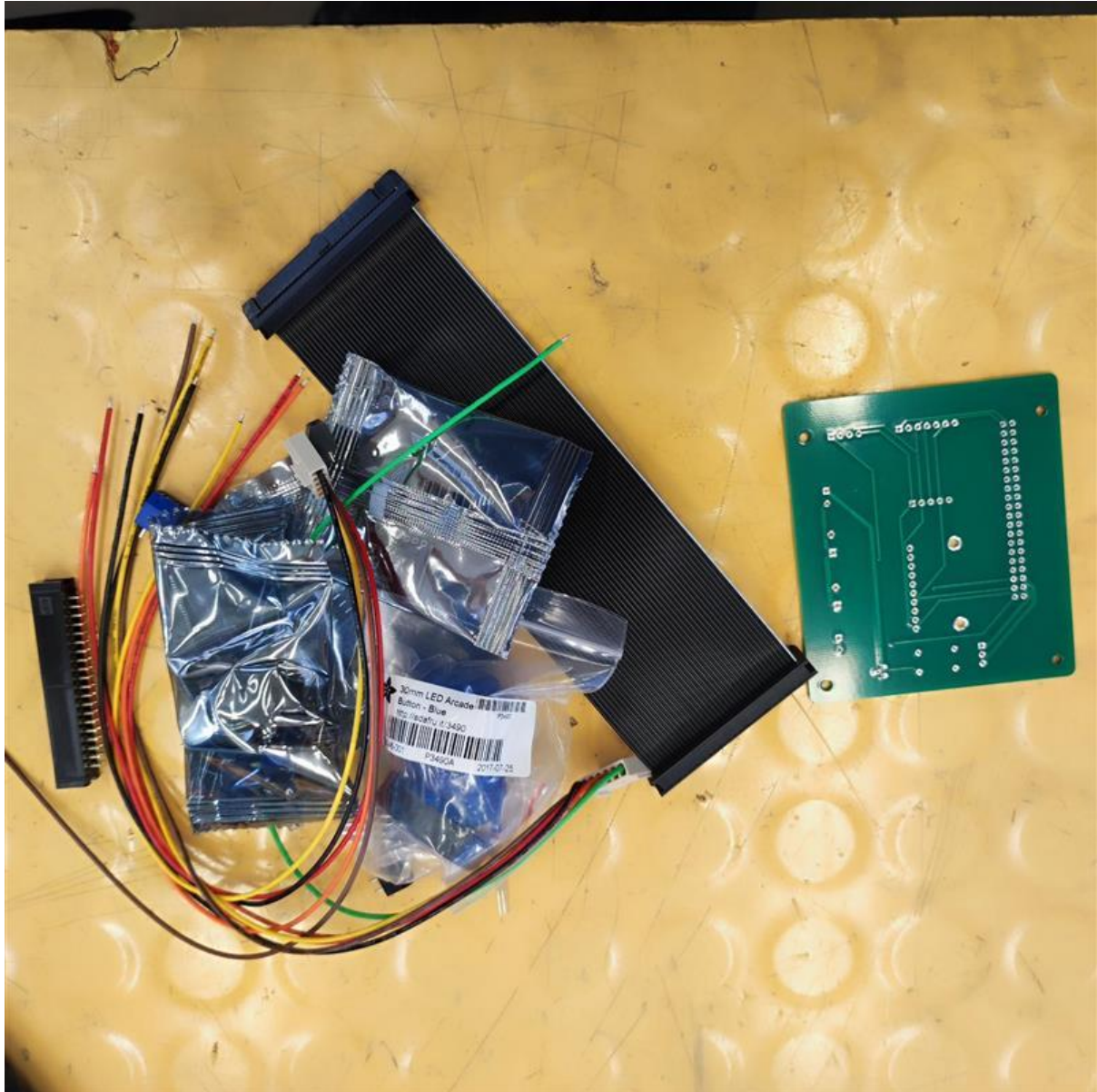
Last step was to add some feet to the bottom of the base plate:



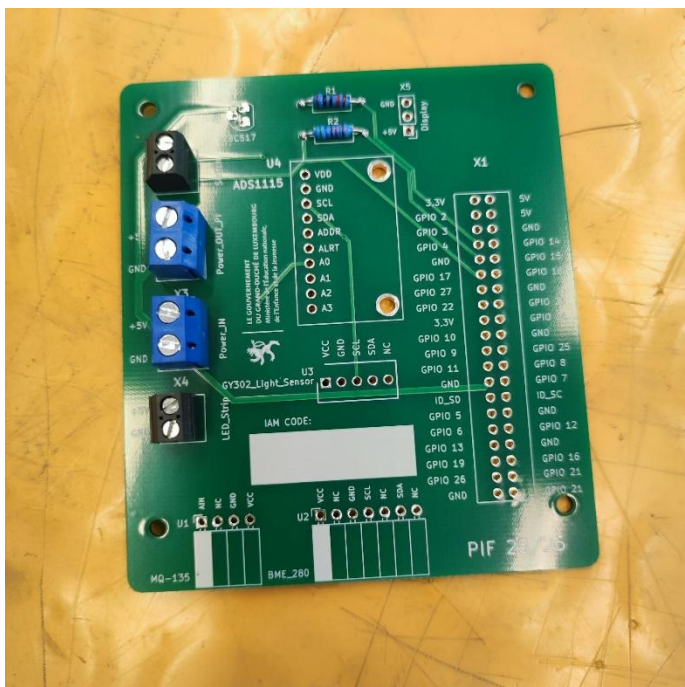
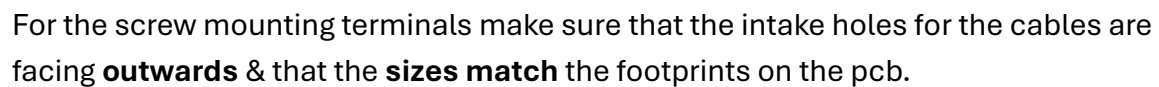
And with all of these steps the basic construction is finished now.

PCB assembly

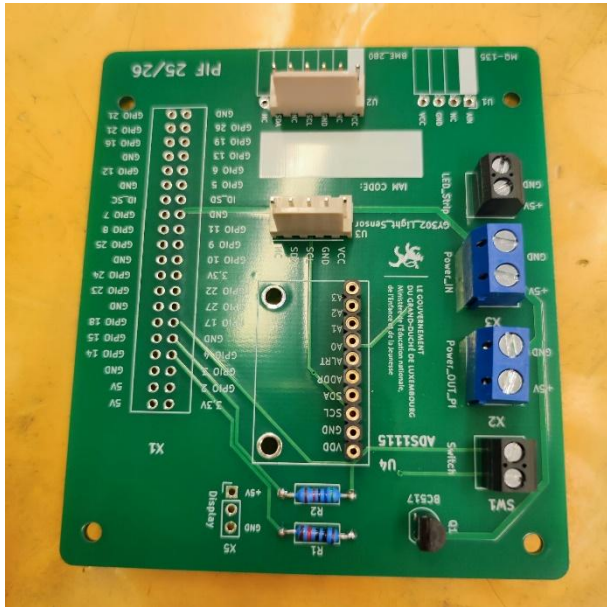
Check that all your components are present and are correct according to the material list:



NOTE: Double check that the resistance of the resistors matches the schematics resistance. It is strongly recommended to check this with a multimeter.



The next step is to solder both JST connectors. You may notice that the connector for U2 or **MBE280** do not match the pin number of the connector. Make sure that the pin **NC (Not connected)** on the **U2** value does not have a pin and that **VCC** gets this pin. The orientation of both jst connectors must face to the “**south**” of the board like shown in the picture. Lastly make sure that for the **transistor** soldered onto the pcb the flat side on the back of the component matches the pcb’s footprint



Now for the **ADS1115** solder it to the pinheader provided in the package, solder the pin socket on the right place, push it into the socket and make sure everything is straight.



Now solder the last components which is a 3 pin header for the display, a 4 pin socket for the **MQ-135** & a 40 pin header for the **GPIO pins** of the **raspberry pi**. You may notice that the 40 pin header has a tiny **hole** on one side of the connector in the middle. This **hole** must face inwards in the pcb like shown in the photo.



Before mounting it don't forget to write your IAM code in the blank part where it is written **IAM CODE**:



You can also connect all the for the moment available cables onto it for the moment:



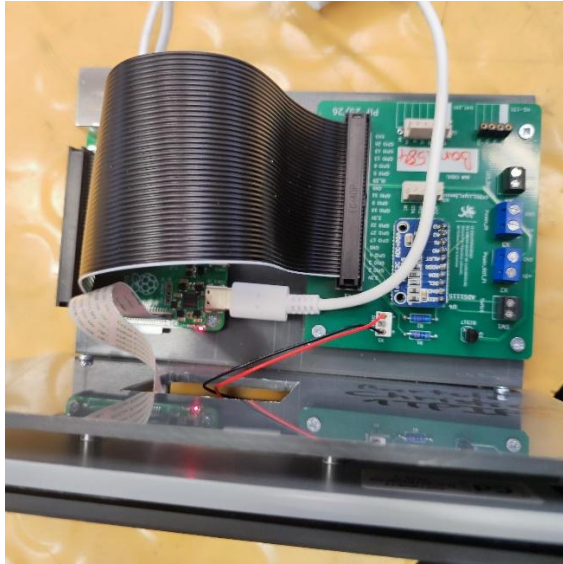
The next thing to do is to screw it onto the base PVC plate into the appropriate drilled holes drilled earlier and use some M2.5 screws with a washer & on top of the washer a nut on each one.



Now connect the big ribbon cable to the raspberry pi & reconnect the display's cable to the pin header on the top right of the PCB where it is written **Display**

To test that everything works plug in the **UBC-C PSU** as well.

NOTE: It is highly recommend to disconnect the JST cables to prevent any short circuits as the cables from the connector can short together as they do not have an insulation right know.

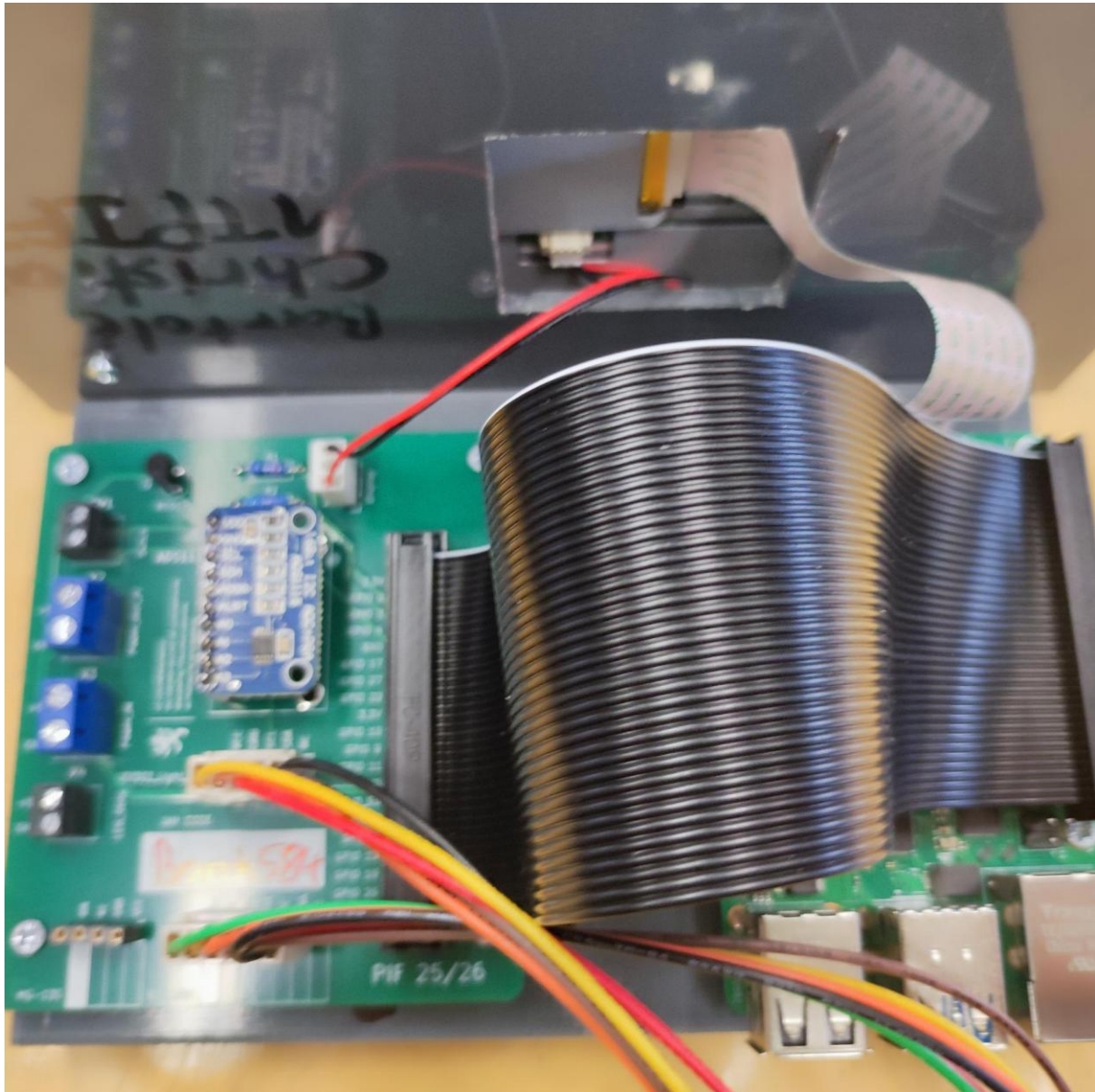


It should now turn on everything when providing electricity.

NOTE: The connector for the power of the display has pretty loose pins which means that you MAY have to adjust the connector on the pin header a bit with your fingers to get the display working



After disconnecting the power you can also replugin the JST cables back into their corresponding JST pin headers.



And after all of that the pcb is complete

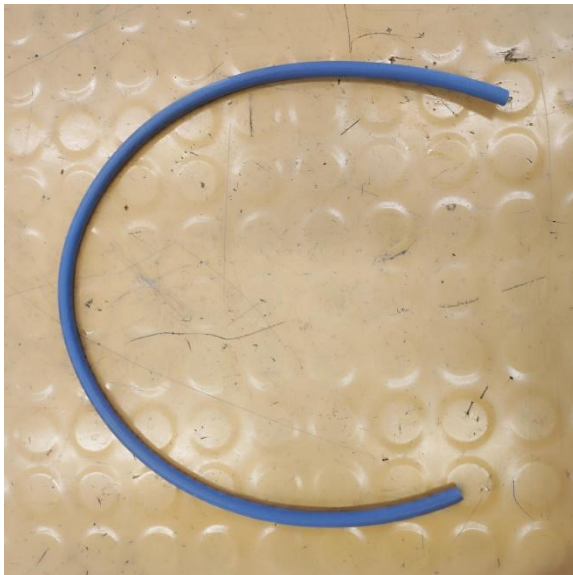
Cable repair

Starting note:

This documentation will not be about repairing a raspberry pi cable but about a generic cable repair that can be used for practically almost any cable. Almost all the steps for a raspberry PI cable repair are the same. This documentation will cover removing a cable insulation, wire stripping and connecting the wires all back together with proper insulation with a soldering iron and a heat gun.

Uninsulate wires

We'll start off with a simple wire like this one:



And we will cut it in half to about the same size on each end:



Next step is to uninsulate the cable. Make sure that you do not damage the wires while cutting in the rubber layer keeping all wires together. There are many tools to do this, but the most commons are a wire stripping knife or a rotary wire stripper Take all the shield of the cable, “group” it together and twist it to get from this:



To this:



There is in this cable one more insulating rubber layer which also needs to be taken off to get to the individual cables like shown below:

NOTE: a lot of cables only have one rubber layer and then already the individual wires and some also do not even have a shielding. This heavily depends on the cable you use so depending on that some steps do not need to be made.

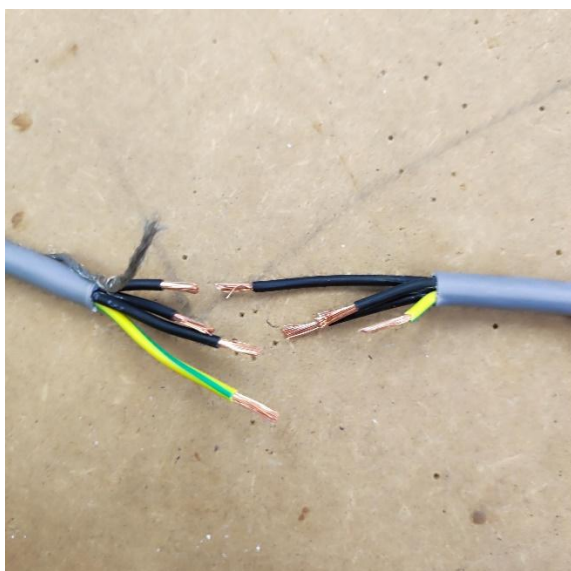
In this cable there is one more rubber insulation under the shield which leads to the actual individual wires. Again, while taking the insulation of make sure that you do **not** damage any wire.



Wire stripping

Afterwards to continue the repair use a wire stripper (again) to expose the metal of each cable like in the photo below. Make sure that you destroy the least of those “tiny” metal cables as possible as this can easily happen with the wire stripper if too much force is applied.

Also staggering them like in the photo below, where you can e.g. see that both wires of the earth do not have the same length to improve the cables stability overall. This must be done for each one of course and as noticed each one should have a different length on both ends.



Connect each wire back together

Now to get the wires back together twist the ends of the exposed wires to get it more compact and apply enough solder to each end:



Before soldering & thereby connecting everything with the soldering iron, get the insulation already on each wire. After which put the insulating part enough to one end so it does not burn under the soldering iron. This process must of course be repeated for each individual wire.

NOTE: Make sure to use max heat (usually around 480-500°C) and take enough time to get a sturdy connection



Cable insulation

Next get the cable insulation for the cable over the cable and make sure everything is covered in it and no end of any wire is exposed. Afterwards apply heat with a heat gun and your cables are connected securely.



Shielding tips

Of course, do not forget to repeat the moves for the earth as well. For the earth I highly recommend twisting both ends together to make the repair much easier, when soldering afterwards and do not forget to put an insulation for it on there as well.

NOTE: you should use a longer piece of cable insulation so that even if one cable is not properly insulated, it does not interfere directly with the shielding.

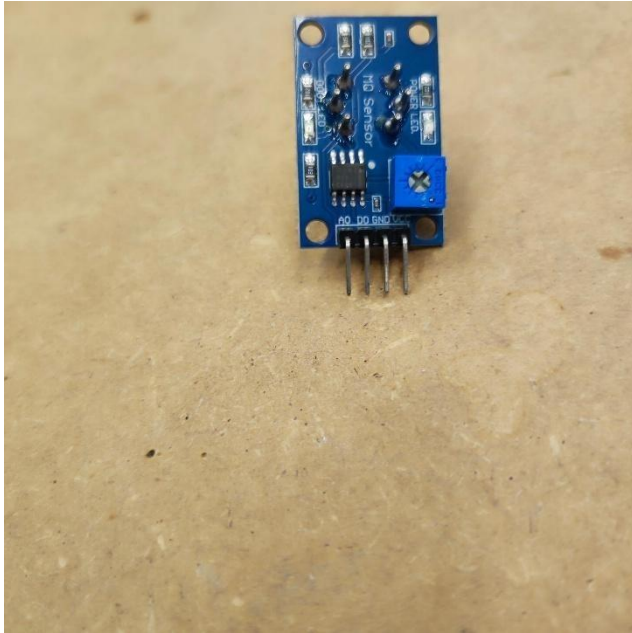


Sensors 1

Gas sensor

Take out the gas sensor and check for

Take out the gas sensor and check for the pinout on the component and the pcb.



Afterwards plug it in according to the pins on the PCB. The sensor must face inwards like shown in the photo:



Bit I2C converter

Now for the 16-bit i2c converter simply solder the pinheader to the component like shown in the photo below:

NOTE: Make sure that the pinheaders are straight. This can be achieved by starting to solder 1 pin and if it is not straight, go over it with soldering iron and straighten it. Afterwards proceed with soldering the remaining pins.



Now plug it into the right slot on the pcb with the pinout matching on both sides (PCB & the component).

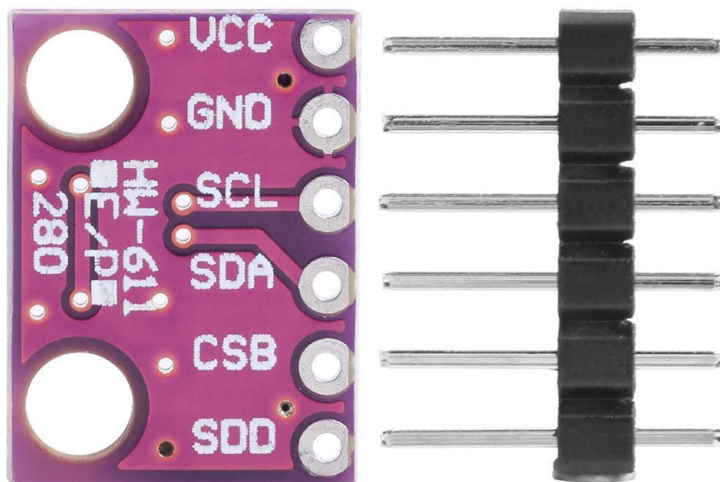


Sensors 2

Biometric sensor

Take out the biometric sensor (HW-611) and check for the pinout on the component and the pcb. Afterwards solder the wires to the pins according to the schematic.

Make sure you take the right connector as both are different in number of pins (for this one you need the 6 pin connector, including **NC**)



1

Black wire -> VCC

Red wire -> GND

Orange wire -> SCL

Green wire -> SDA

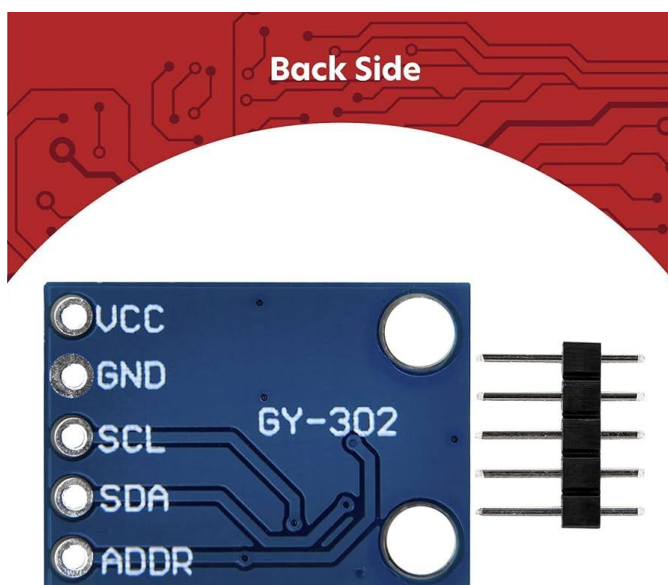
CSB & SDD are not in use and can be left unconnected. The two (2) wires (which are **brown** and **yellow**) should be cut to the same length with a wire cutter the to only expose the end of both wires. Next, take a piece of wire insulation and stick it over both wires. Lastly heat it (with e.g. a heating gun) and crimp them together with a gripper (or any other crimping tool that works for this case).

Make sure to not solder any pin on the component to **NC** as this is simply **Not Connected** to anything.

Gyroscope sensor

Now for the gyroscope sensor (GY-302), take it out and check for the pinout on the component and the pcb. Afterwards solder the wires to the pins according to the schematic.

Make sure you take the right connector as both are different in number of pins (for this one you need the 5 pin connector, including **NC**)



Black wire -> VCC

Brown wire -> GND

Red wire -> SCL

Orange wire -> SDA

Yellow wire -> ADDR

Sensor programming (**BME280 / HW-611**)

First start by installing python with **sudo apt install python3-venv python3-full python3-vnv**

```
barch584@PIbarch584:/opt/station/Firmware_und_Scripts $ sudo apt install python3-venv python3-full
python3-venv is already the newest version (3.13.5-1).
Installing:
  python3-full

Installing dependencies:
  fonts-mathjax      libpython3.13-testsuite  python3.13-doc
  idle               python3-doc              python3.13-examples
  idle-python3.13    python3-examples         python3.13-full
  libjs-mathjax      python3-gdbm             python3.13-gdbm

Suggested packages:
  fonts-mathjax-extras  fonts-stix  libjs-mathjax-doc  python3-gdbm-dbg

Summary:
  Upgrading: 0, Installing: 13, Removing: 0, Not Upgrading: 288
  Download size: 29.4 MB
  Space needed: 155 MB / 21.6 GB available

Continue? [Y/n] y
```

Next navigate to **/opt/station** with **cd /opt/station** and run **python3 -m venv venv**

```
barch584@PIbarch584:/opt/station $ sudo python3 -m venv venv
```

Activate the venv mode with **source venv/bin/activate**.

Afterwards install the extra needed python libraries for i2c devices, for which the command **sudo apt install -y python3-smbus i2c-tools** is needed

```
barch584@PIbarch584:/opt/station $ sudo apt install -y python3-smbus i2c-tools
```

Next, install the library for the bmp280 for which simply run **pip install bmp280**. Also ensure to install for bme280 with **pip install bme280**, as this at least for me created problems without it during testing.

```
barch584@PIbarch584:/opt/station $ pip install bmp280
```

If the previous command does not work (due to permission errors) follow this:

Go to /opt with **cd /opt** and give yourself ownership of /station to be able to have writing permissions to avoid permission errors. For that type in **sudo chown -R**

YOURUSERNAME:root station/ which would be in my case **sudo chown -R barch584:root station/**

```
(venv) barch584@PIbarch584:/opt $ sudo chown -R barch584:root station/
```

After successfully installing the libraries modify the **test-bmp280.py** in the firmware & scripts folder with the following:

Source code:

```
from bmp280 import BMP280
from smbus2 import SMBus
import time
```

```
ADDRESS = 0x76
```

```
def main():
```

```
    bus = SMBus(1)
```

```
    sensor = BMP280(i2c_dev=bus, i2c_addr=ADDRESS)
```

```
    while True:
```

```
        temperature = sensor.get_temperature()
```

```
        pressure = sensor.get_pressure()
```

```
        print(f"Temperature: {temperature:.2f} °C")
```

```
        print(f"Pressure: {pressure:.2f} hPa")
```

```
        print("-" * 30)
```

```
        time.sleep(2)
```

```
if __name__ == "__main__":  
    main()
```

NOTE: the **ADDRESS** variable may be different. To check for the correctly one unplug all the sensors except the barometric sensor and run **sudo i2cdetect -y 1** and check for any number in the entire table. (Usually the address for this sensor is **76**, but this may change)

Now run the bmp script and you should see measurements:

python3 Firmware_und_scripts/test-bmp280.py

NOTE: I changed the foldername of Firmware und scripts so double check that the folder name in your case is correct.

```
(venv) barch584@PIbarch584:/opt/station $ python3 Firmware_und_Scripts/test-bmp280.py  
Temperature: 24.94 °C  
Pressure: 995.87 hPa  
-----  
Temperature: 24.93 °C  
Pressure: 995.87 hPa  
-----  
Temperature: 24.93 °C  
Pressure: 995.87 hPa  
-----  
Temperature: 24.92 °C  
Pressure: 995.86 hPa  
-----
```

Sensor programming (BH1750)

In your firmware and scripts folder simply start by editing the test-bh1750.py with **sudo nano test-bh1750.py** and change it to the following code:

```
import smbus  
import time
```

```
DEVICE = 0x23 # Your detected address
POWER_ON = 0x01
RESET = 0x07
CONTINUOUS_HIGH_RES_MODE = 0x10
```

```
bus = smbus.SMBus(1)
```

```
def read_light():
    bus.write_byte(DEVICE, POWER_ON)
    bus.write_byte(DEVICE, RESET)
    bus.write_byte(DEVICE, CONTINUOUS_HIGH_RES_MODE)
    time.sleep(0.2)

    data = bus.read_i2c_block_data(DEVICE, CONTINUOUS_HIGH_RES_MODE, 2)
    result = (data[0] << 8) | data[1]
    lux = result / 1.2
    return lux
```

```
while True:
    try:
        lux = read_light()
        print("Light Level: {:.2f} lux".format(lux))
        time.sleep(1)
    except Exception as e:
        print("Error:", e)
        time.sleep(1)
```

This version has a while loop to get continuous values (for the documentation specifically). You may also need to change the i2c address, meaning the value of the variable **DEVICE**.

Afterwards save it and run it with **python3 test-bh1750.py**

This value can be manipulated by e.g. putting a finger over the sensor or by holding a torch over the sensor.

```
barch584@PIbarch584:/opt/station/Firmware_und_Scripts $ python3 test-bh1750.py
Light Level: 388.33 lux
Light Level: 390.00 lux
Light Level: 390.83 lux
Light Level: 393.33 lux
Light Level: 395.00 lux
Light Level: 390.00 lux
Light Level: 290.00 lux
Light Level: 325.83 lux
Light Level: 342.50 lux
Light Level: 7.50 lux
Light Level: 24.17 lux
Light Level: 65.83 lux
Light Level: 70.83 lux
Light Level: 185.83 lux
Light Level: 180.83 lux
```

Sources (for sprint Sensors 2):

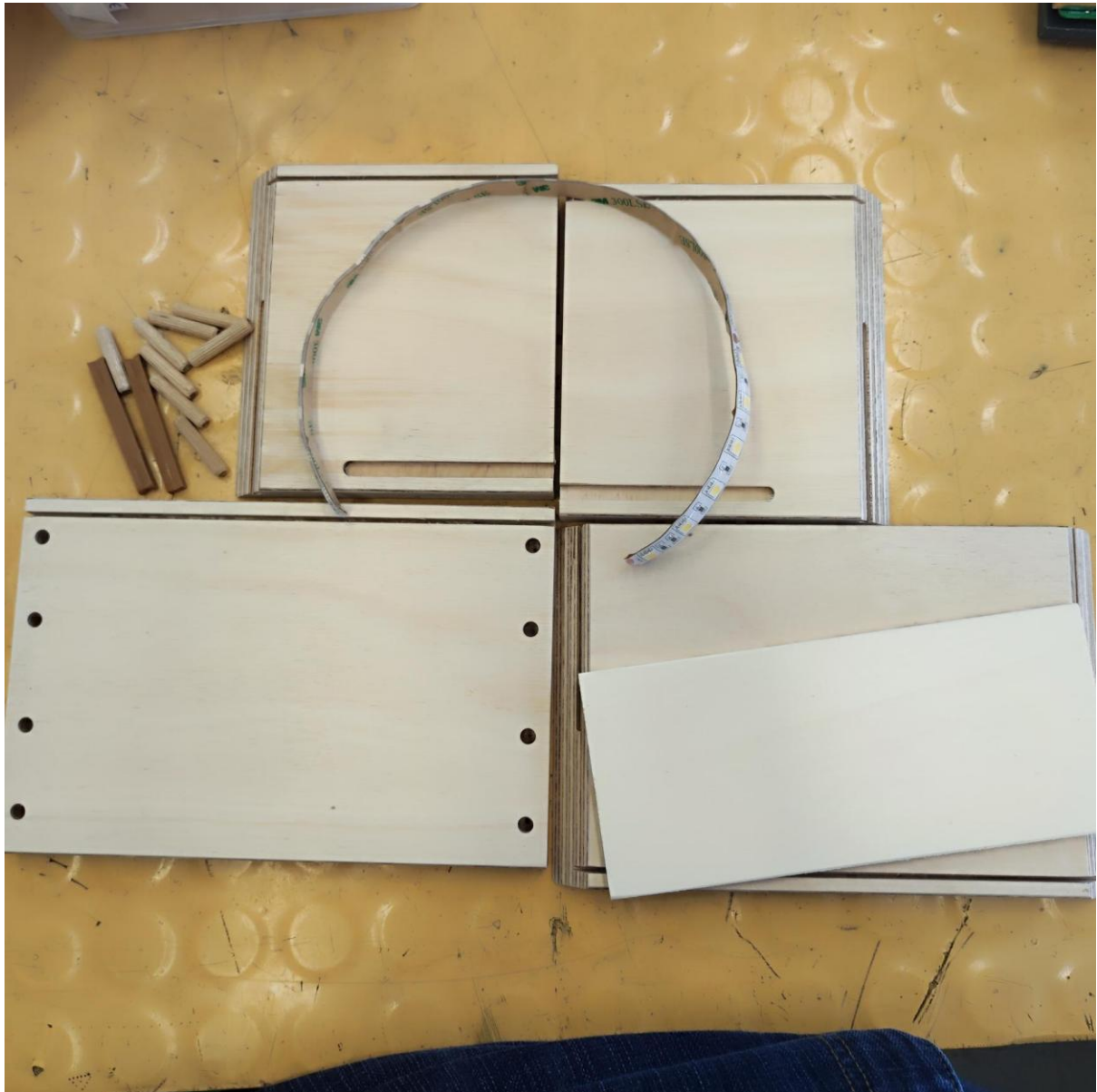
¹ <https://www.joom.com/de/products/64cf5e1c3899ad01e54e230a>

² <https://www.amazon.de/AZDelivery-GY-302-BH170-Sensor-Parent/dp/B07TKWNGZ4?th=1>

Assembly

Material

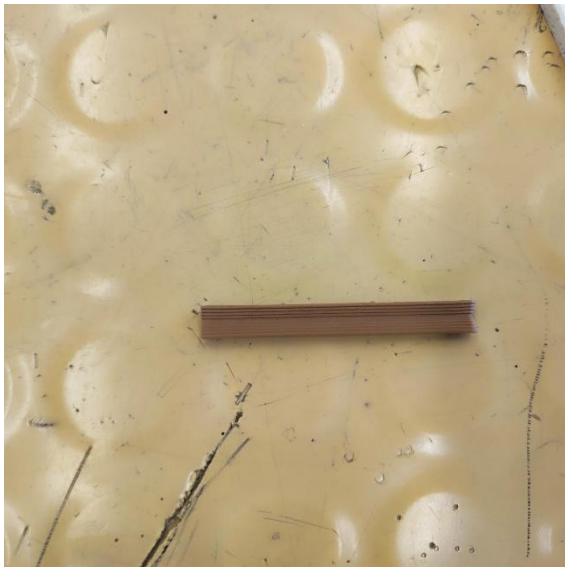
Here is everything you will need for this sprint:



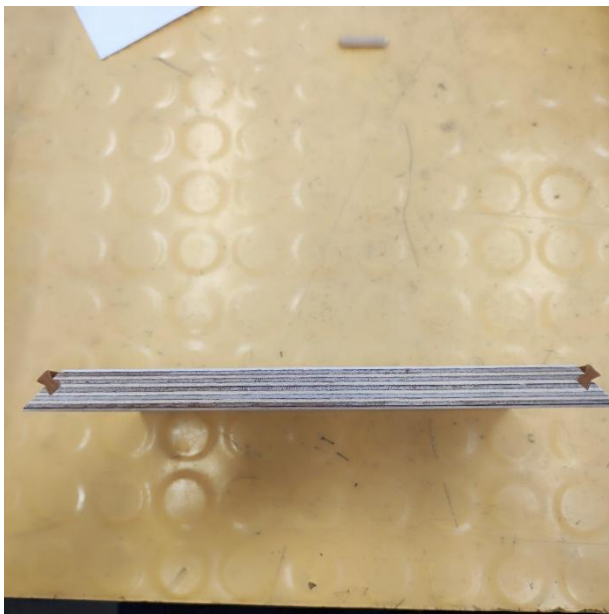
Assembly (wooden box)

Start by getting the base plate and attach these flexible bracket to them. (This might take some force to complete)

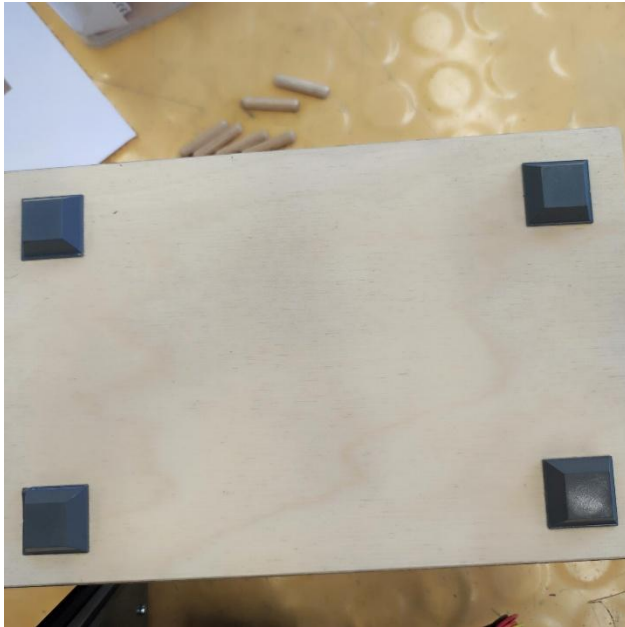
NOTE: Pay attention which end goes where. The rounded part of the bracket on the end of it goes inside and the more corner end must face outward:



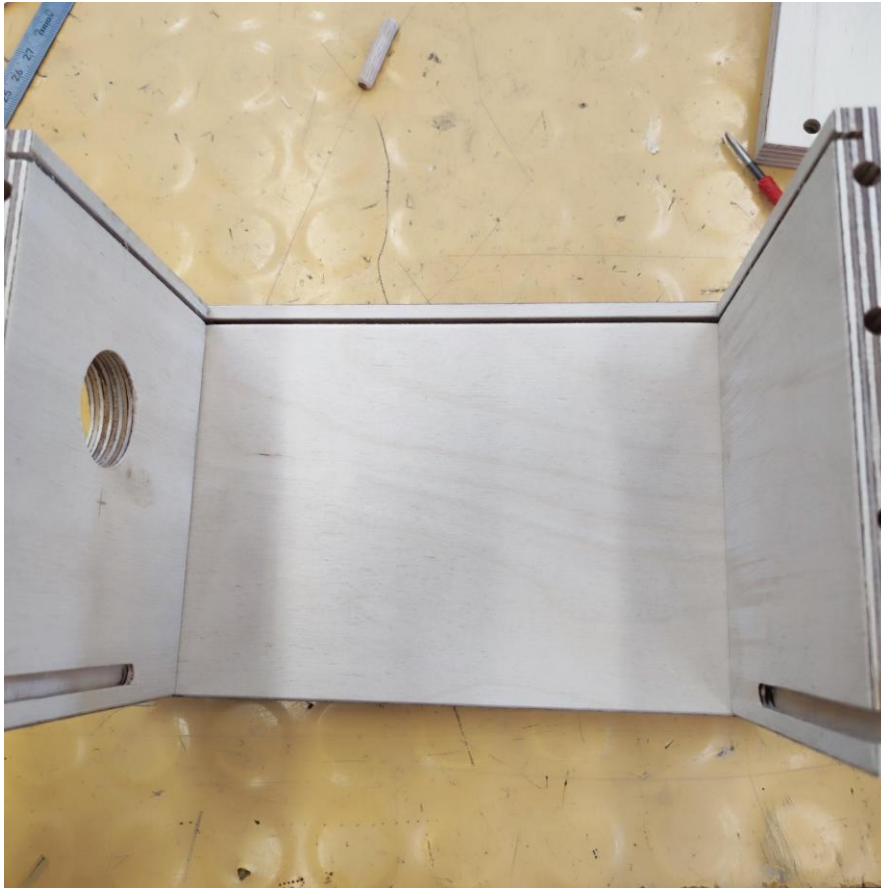
It should look something like this:



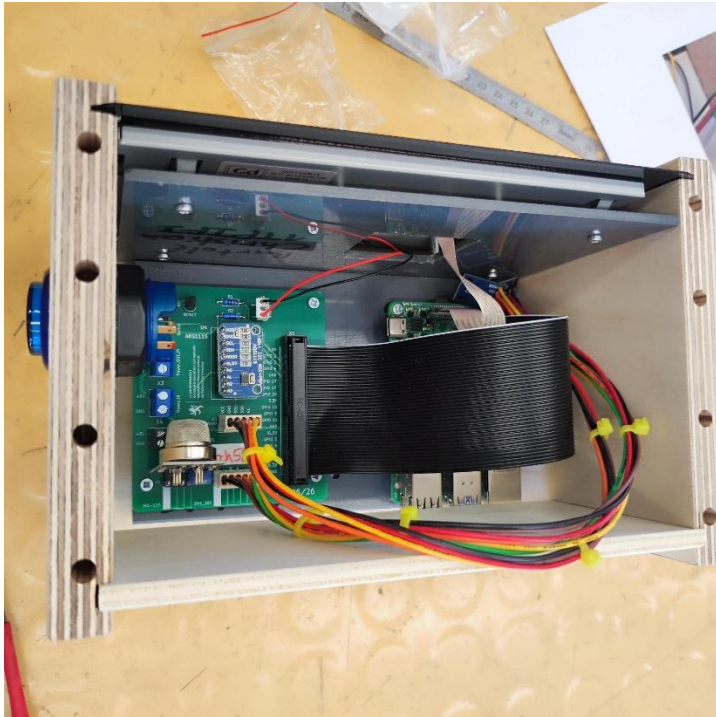
Remove also the feet from the PVC plates and repute them to the bottom part of the wooden box:



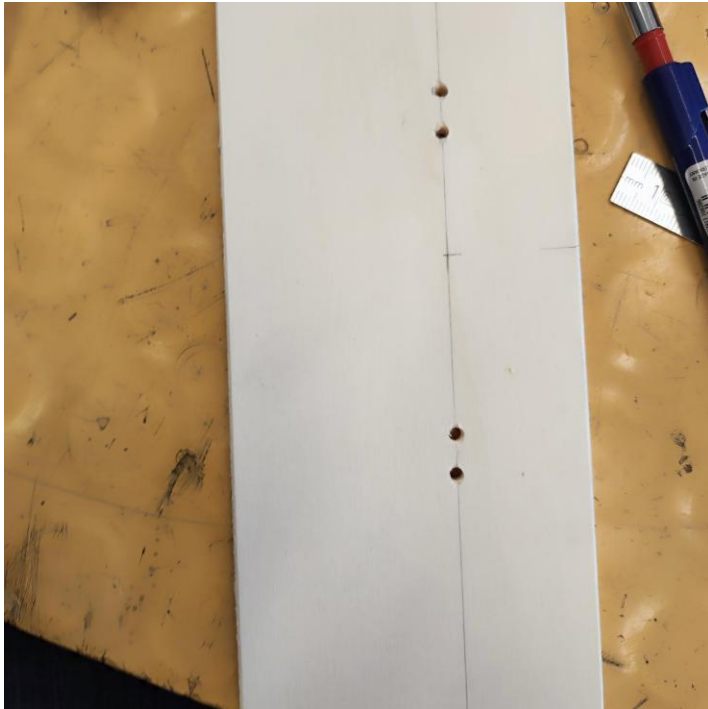
Now slide each side to the brackets on the bottom plate:



Afterwards put the PVC base with the raspberry PI, the custom pcb with the display together in the box like this:



Now for the back plate simply put some holes in it for the sensors.



Also don't forget to screw your sensors to it.

Assembly (button)

For this you need 2 wires (preferably in red & black)

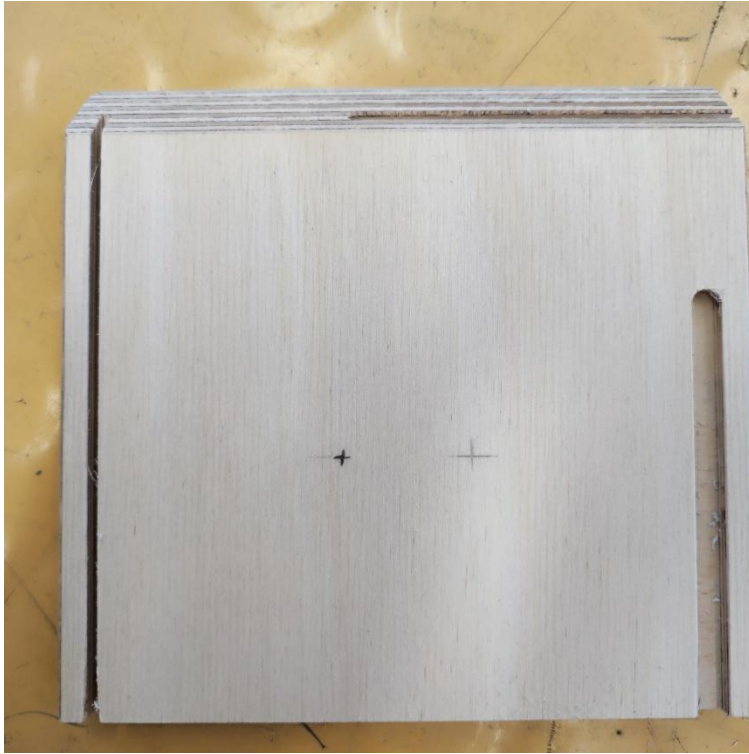


Start by uninsulating each end of them:



Afterwards make yourself a marking for drilling.

NOTE: If you assembled everything already together you may have to take apart everything to drill this needed hole for the button:



And start by drilling with a step bit to open it up

NOTE: To achieve a better result, it is highly recommended to pre-drill the hole with a smaller drill before drilling with the step bit.



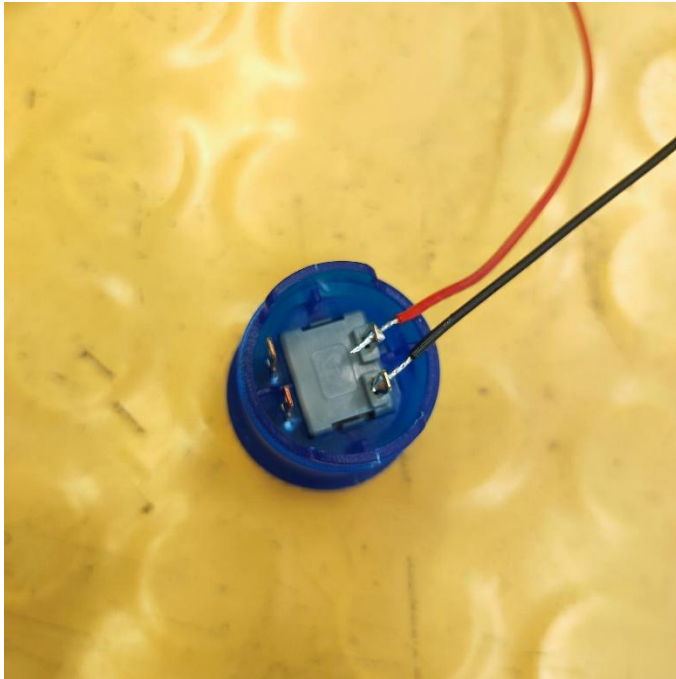
Then start by putting your button inside of it and fixing it in place with the rubber ring:



And it should look something like this:

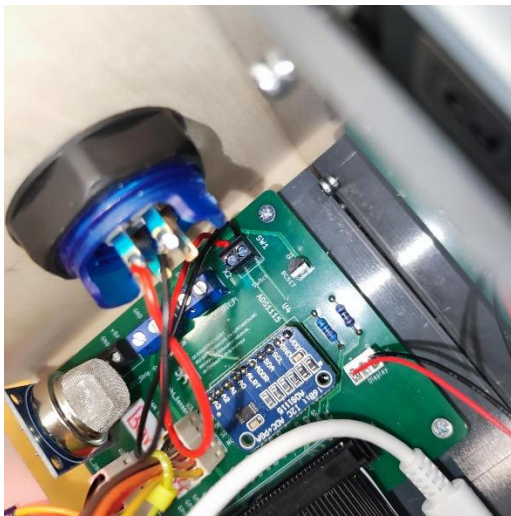


The cables must be soldered to the contacts of this gray sort of “box” like shown below. As it is a button with no polarity it does not matter if the red wire is left or right and vice versa. Only make sure that it is not connected to the 2 other pins:



After having it put in place connect both wires to the screw-terminal where it is written **SW1** on the PCB. Again the polarity does not matter in this case.

NOTE: This is the black smaller part with the 2 screws on top.



LED strip

Preparing LED strip

Start with 2 pieces of wire (black & red is recommended)



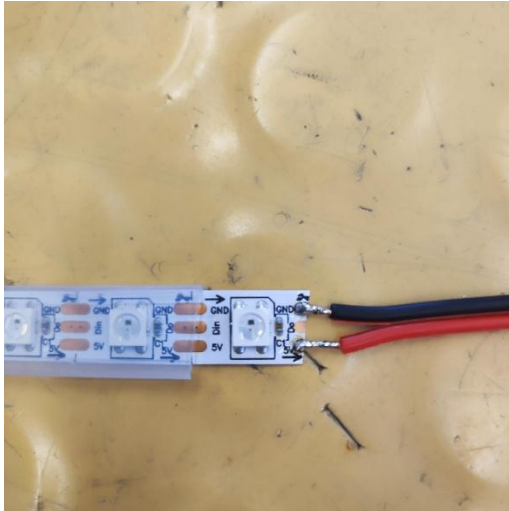
Now take of the insulation with a wire stripper. You could also already apply some solder to each wire to make the connection later on easier like shown below:



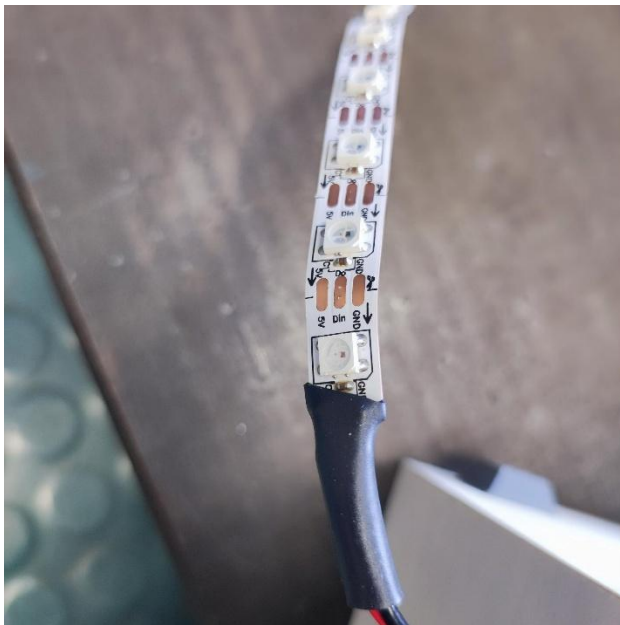
Now it is time to solder both wires to the LED strip. I highly recommend to take the strip a bit of its plastic holder like here before soldering the wires to make your life easier:

Black -> GND

Red -> 5V

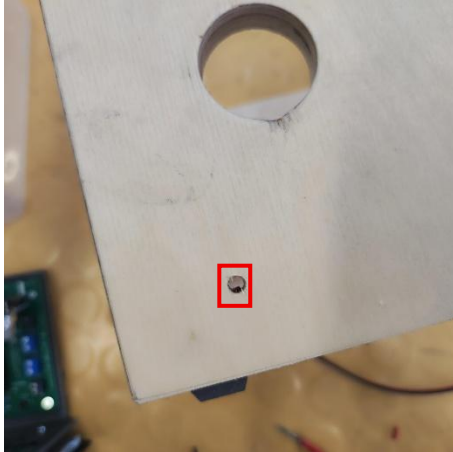


Also do not forget to apply a heatshrink to make a secure connection. A heating gun must be used to shrink the actual tubing:



Putting the LED strip in place

Now before putting the strip in place a hole must be drilled on the side where the hole for the button was made in previous sprints. A 3mm should do the job:



Now let's glue the actual strip to the front. For that I used a hot glue gun and applied enough glue to it :

NOTE : The plastic cover of the LED strip must be removed otherwise the glue will not hold it properly



Wire the LED strip to the board

After connecting everything the pin layout is the following:

+5V -> Red wire

GND -> Black wire

